

人間計算可能なパスワード認証に対する
畳み込みニューラルネットワークを用いた
安全性評価

1TE21865E 小川 翔平

令和 7 年 2 月

九州大学 工学部 電気情報工学科

概要

Manuel Blum らによって提案された人間計算可能なパスワードとは、ユーザが暗算できる秘密関数をもとに秘密計算を行い、その結果を利用して認証を行うチャレンジレスポンス認証方式の一種である。この方式では、認証ごとに異なるパスワードを生成／入力できるため、パスワードの使い回しを防ぎ、情報漏洩時のリスクを低減できるとされている。

一方で、既存研究では、認証時に生成／入力されたチャレンジ-レスポンスの組み合わせが流出した場合、機械学習を用いて認証情報を推測できる可能性が指摘されている。これまでの研究では、多層パーセプトロン（MLP）や再帰型ニューラルネットワーク（RNN）を用いた解析が行われてきたが、いずれも予測精度が低く、現実的に認証情報を推測することは困難であるとされていた。

本研究では、まず既存研究において比較的高い予測精度を示した手法の再現を行い、その結果をもとに新たなモデルの提案と評価を実施する。具体的には、埋め込み層を前処理として導入した双方向長短期記憶ニューラルネットワーク（Bi-LSTM）を再現し、その予測精度を検証する。さらに、既存手法の課題を踏まえ、埋め込み層を前処理とした畳み込みニューラルネットワーク（CNN）を導入し、機械学習による予測精度向上の可能性を検討する。予測対象とする秘密関数は、次のとおりである：

$$f_{2,2}(x_0, \dots, x_{13}) = x_j + x_{12} + x_{13} \bmod 10, \quad j = x_{10} + x_{11} \bmod 10$$

本研究の結果、再現実験における予測精度は最大で 51.22% を記録し、既存研究（予測精度は最大で 55.81%）とほぼ同様の結果となることを確認した。また、埋め込み層を前処理とした CNN による予測では、最大 60.84% の予測精度を達成し、既存研究からさらに精度を向上することに成功した。

Abstract

The human-computable password proposed by Manuel Blum et al. is a type of challenge-response authentication method in which the user performs secret computations based on a mental secret function and uses the result for authentication. This approach allows for the generation and input of different passwords for each authentication attempt, preventing password reuse and reducing the risk of information leakage.

On the other hand, existing research has pointed out that if the challenge-response pairs generated and input during authentication are leaked, machine learning could potentially be used to infer authentication information. Previous studies have analyzed this issue using multilayer perceptrons (MLP) and recurrent neural networks (RNN). However, their prediction accuracy was low, making it practically difficult to infer authentication information.

In this study, we first reproduce the method that showed relatively high prediction accuracy in existing research and use the results as a basis for proposing and evaluating a new model. Specifically, we reproduce a bidirectional long short-term memory (Bi-LSTM) neural network with an embedding layer as preprocessing and verify its prediction accuracy. Furthermore, considering the limitations of existing methods, we introduce a convolutional neural network (CNN) with an embedding layer as preprocessing and examine the potential for improving prediction accuracy using machine learning. The secret function to be predicted is as follows:

$$f_{2,2}(x_0, \dots, x_{13}) = x_j + x_{12} + x_{13} \bmod 10, \quad j = x_{10} + x_{11} \bmod 10$$

As a result of this study, the reproduction experiment achieved a maximum prediction accuracy of 51.22%, confirming that the results were almost the same as those of existing research (which recorded a maximum accuracy of 55.81%). Additionally, the prediction using a CNN with an embedding layer as preprocessing achieved a maximum accuracy of 60.84%, successfully improving accuracy beyond previous studies.

目次

概要	i
Abstract	ii
図目次	v
表目次	vii
第 1 章 緒論	1
1.1 研究背景	1
1.2 本研究の情報学的な貢献	2
第 2 章 背景知識	3
2.1 認証	3
2.1.1 認証とは	3
2.1.2 様々な認証方式	3
2.1.3 パスワード認証	4
2.1.4 チャレンジレスポンス認証	5
2.2 機械学習	5
2.2.1 機械学習とは	5
2.2.2 様々な機械学習モデル	6
2.2.3 埋め込み層	7
2.2.4 長短期記憶	7
2.2.5 畳み込みニューラルネットワーク	8
第 3 章 既存研究	9
3.1 人間計算可能なパスワード	9
3.1.1 人間計算可能なパスワードとは	9
3.1.2 人間計算可能の定義	9
3.1.3 一桁チャレンジ	10
3.1.4 人間計算可能関数	12
3.1.5 認証プロセス	12
3.2 埋め込み層および双方向長短期記憶による安全性評価	13
第 4 章 研究手法	15
4.1 本研究の研究内容	15
4.2 実験手法	15
4.2.1 タスク定義とモデルの設計	15

4.2.2	実験に使用するデータセット	16
4.2.3	実験に使用する機械学習モデル	16
4.2.4	実験の手順	17
 第5章 結果		18
5.1	埋め込み層および双方向長短期記憶を用いた実験結果	18
5.2	埋め込み層および畳み込みニューラルネットワークを用いた実験結果	23
5.2.1	$(k_1, k_2) = (2, 2)$ における実験結果	24
5.2.2	$(k_1, k_2) = (1, 3)$ における実験結果	30
 第6章 考察		37
 第7章 結論		38
 謝辞		39
参考文献		40
付録: プログラムコード		41

図目次

図 3.1.3-1	一桁チャレンジにおいて記憶するマッピング関数 σ とその計算例	11
図 4.2.2-1	実験に使用するデータセットの例 ($N=100$ の場合)	16
図 5.1-1	$N=26$, データ数 1000 における Bi-LSTM を用いた予測精度	18
図 5.1-2	$N=26$, データ数 5000 における Bi-LSTM を用いた予測精度	18
図 5.1-3	$N=26$, データ数 10000 における Bi-LSTM を用いた予測精度	19
図 5.1-4	$N=26$, データ数 50000 における Bi-LSTM を用いた予測精度	19
図 5.1-5	$N=50$, データ数 1000 における Bi-LSTM を用いた予測精度	20
図 5.1-6	$N=50$, データ数 5000 における Bi-LSTM を用いた予測精度	20
図 5.1-7	$N=50$, データ数 10000 における Bi-LSTM を用いた予測精度	21
図 5.1-8	$N=50$, データ数 50000 における Bi-LSTM を用いた予測精度	21
図 5.1-9	$N=100$, データ数 1000 における Bi-LSTM を用いた予測精度	22
図 5.1-10	$N=100$, データ数 5000 における Bi-LSTM を用いた予測精度	22
図 5.1-11	$N=100$, データ数 10000 における Bi-LSTM を用いた予測精度	23
図 5.1-12	$N=100$, データ数 50000 における Bi-LSTM を用いた予測精度	23
図 5.2.1-1	$(k_1, k_2) = (2, 2)$ 型: $N=26$, データ数 1000 における CNN を用いた予測精度	24
図 5.2.1-2	$(k_1, k_2) = (2, 2)$ 型: $N=26$, データ数 5000 における CNN を用いた予測精度	25
図 5.2.1-3	$(k_1, k_2) = (2, 2)$ 型: $N=26$, データ数 10000 における CNN を用いた予測精度	25
図 5.2.1-4	$(k_1, k_2) = (2, 2)$ 型: $N=26$, データ数 50000 における CNN を用いた予測精度	26
図 5.2.1-5	$(k_1, k_2) = (2, 2)$ 型: $N=50$, データ数 1000 における CNN を用いた予測精度	26
図 5.2.1-6	$(k_1, k_2) = (2, 2)$ 型: $N=50$, データ数 5000 における CNN を用いた予測精度	27
図 5.2.1-7	$(k_1, k_2) = (2, 2)$ 型: $N=50$, データ数 10000 における CNN を用いた予測精度	27
図 5.2.1-8	$(k_1, k_2) = (2, 2)$ 型: $N=50$, データ数 50000 における CNN を用いた予測精度	28
図 5.2.1-9	$(k_1, k_2) = (2, 2)$ 型: $N=100$, データ数 1000 における CNN を用いた予測精度	28
図 5.2.1-10	$(k_1, k_2) = (2, 2)$ 型: $N=100$, データ数 5000 における CNN を用いた予測精度	29
図 5.2.1-11	$(k_1, k_2) = (2, 2)$ 型: $N=100$, データ数 10000 における CNN を用いた予測精度	29
図 5.2.1-12	$(k_1, k_2) = (2, 2)$ 型: $N=100$, データ数 50000 における CNN を用いた予測精度	30
図 5.2.2-1	$(k_1, k_2) = (1, 3)$ 型: $N=26$, データ数 1000 における CNN を用いた予測精度	31
図 5.2.2-2	$(k_1, k_2) = (1, 3)$ 型: $N=26$, データ数 5000 における CNN を用いた予測精度	31
図 5.2.2-3	$(k_1, k_2) = (1, 3)$ 型: $N=26$, データ数 10000 における CNN を用いた予測精度	32
図 5.2.2-4	$(k_1, k_2) = (1, 3)$ 型: $N=26$, データ数 50000 における CNN を用いた予測精度	32
図 5.2.2-5	$(k_1, k_2) = (1, 3)$ 型: $N=50$, データ数 1000 における CNN を用いた予測精度	33
図 5.2.2-6	$(k_1, k_2) = (1, 3)$ 型: $N=50$, データ数 5000 における CNN を用いた予測精度	33
図 5.2.2-7	$(k_1, k_2) = (1, 3)$ 型: $N=50$, データ数 10000 における CNN を用いた予測精度	34
図 5.2.2-8	$(k_1, k_2) = (1, 3)$ 型: $N=50$, データ数 50000 における CNN を用いた予測精度	34
図 5.2.2-9	$(k_1, k_2) = (1, 3)$ 型: $N=100$, データ数 1000 における CNN を用いた予測精度	35

- 図 5.2.2-10 $(k_1, k_2) = (1, 3)$ 型 : $N = 100$, データ数 5000 における CNN を用いた予測精度 35
- 図 5.2.2-11 $(k_1, k_2) = (1, 3)$ 型 : $N = 100$, データ数 10000 における CNN を用いた予測精度 36
- 図 5.2.2-12 $(k_1, k_2) = (1, 3)$ 型 : $N = 100$, データ数 50000 における CNN を用いた予測精度 36

表目次

表 3.2-1	埋め込み層および Bi-LSTM の構成	13
表 3.2-2	埋め込み層および Bi-LSTM を用いた予測精度	14
表 4.2.3-1	埋め込み層および CNN の構成	16
表 5.2.1-1	$(k_1, k_2) = (2, 2)$ 型における埋め込み層および CNN を用いた予測精度	24
表 5.2.2-1	$(k_1, k_2) = (1, 3)$ 型における埋め込み層および CNN を用いた予測精度	30

第1章

緒論

1.1 研究背景

近年、さまざまな情報サービスの普及に伴い、ユーザ認証の手法としてパスワード認証が広く利用されている。パスワード認証は、ユーザが各サービスに対して独自の秘密の文字列を設定し、その文字列を毎回入力することによって認証が行われる、いわゆる「知識認証」の一形態である。しかし、現実には、ユーザが管理するサービスの数が増えるにつれて、パスワードを使い回す傾向が強まり、その結果としてセキュリティ上のリスクが生じる。特に、1つのサービスでパスワードが漏洩すると、他のサービスにおけるアカウントも不正アクセスの対象となる可能性が高くなる。このような問題に対して、より安全な認証手法が求められている。

この課題を解決するために、2016年に Manuel Blum らによって提案されたのが「人間計算可能なパスワード」である[1]。この方法は、ユーザが事前に画像と数値の対応関係および計算に用いる関数を記憶し、認証時には画像をもとに計算結果を入力するという仕組みを採用している。画像ごとに異なるパスワードを生成できるため、使い回しが避けられ、万が一情報が漏洩してもセキュリティが保たれる可能性が高い。また、この方式では毎回異なるパスワードが生成されるため、認証のたびに新たなパスワードを入力する必要があり、セキュリティの向上が期待できる。既存研究では、人間計算可能なパスワード認証において、ユーザに提示された画像と入力した答えの組が流出した場合、その情報をもとに機械学習を用いて認証情報を推測できる可能性が指摘されている[4]。これまでは、多層パーセプトロン (MLP) や再帰型ニューラルネットワーク (RNN) などの手法を用いた解析が行われてきたが、結果としてこれらの手法を用いた予測は精度が低く、現実的に認証情報を推測することは困難であるとされていた[4][5]。

本研究では、既存研究において比較的高い予測精度を示した手法の再現を行い、その結果をもとに新たなモデルの提案と評価を実施する。具体的には、埋め込みを前処理とした双方向長短期記憶 (Bi-LSTM) を用いた機械学習モデルを再現し、その予測精度を検証する[6]。さらに、既存手法の課題を踏まえ、埋め込みを前処理とした畳み込みニューラルネットワーク (CNN) を導入し、機械学習による予測精度向上の可能性を検討する。特に、異なるパラメータ設定を持つ人間計算可能なパスワードに対する予測精度の向上を目指し、それぞれにおけるモデルの性能を比較する。また、パラメータが人間計算可能なパスワードの安全性に与える影響を分析し、予測精度とセキュリティのバランスを評価する。これにより、各パラメータ設定がどのように認証の安全性に寄与するのかを明らかにし、より強固なパスワード設計の指針を提供することを目的とする。

1.2 本研究の情報学的な貢献

本研究の情報学的な貢献として、以下の二つの点を考えている。

第一に、機械学習を用いた認証情報の予測精度向上に関する新たなアプローチの提案である。埋め込みを前処理とした畳み込みニューラルネットワーク（CNN）を活用することによって、予測精度の向上を目指す。

第二に、人間計算可能関数におけるパラメータ設定がセキュリティに与える影響の検討である。特に、 $(k_1, k_2) = (2, 2)$ 型および $(k_1, k_2) = (1, 3)$ 型のパラメータ設定における予測精度と安全性のバランスについて分析し、どのような設定がより安全かを示す。これにより、より安全性の高い人間計算可能なパスワード設計における指針を提供できると考えている。

以上のことから、本研究は、機械学習を利用したセキュリティ手法の評価と改善という点において、情報学的な貢献をすることができると考える。

第 2 章

背景知識

2.1 認証

2.1.1 認証とは

認証とは、システムやサービスにアクセスする際に、利用者がそのサービスを利用する正当な権限を持つ人物であることを確認する一連の手続きである。近年、スマートフォンやタブレットをはじめとしたモバイル端末の普及により、様々なデジタルサービスがユーザに提供されるようになった。これらのサービスは、しばしば物理的な対面を伴わず、ネットワーク経由でアクセスされるため、ユーザが意図した人物であるかを確認し、やり取りされる情報が正確かつ安全であることを保障することが求められる。このような背景の中で、認証は、セキュリティを確保し、データの信頼性を維持するために極めて重要な役割を担っている。

認証の重要性は、特に個人情報や金融情報を取り扱うサービスにおいて顕著である。例えば、オンラインバンキングや電子商取引などでは、利用者の本人確認が不十分であると、悪意のある第三者による不正アクセスが発生する可能性が高くなる。したがって、認証はシステムの信頼性を担保するための基盤となり、ユーザとサービス間の信頼を築く重要な要素である。

2.1.2 様々な認証方式

認証方式は、大きく分けて「知識認証」「所有物認証」「生体認証」の 3 種類に分類される。それぞれの方式は、認証の目的に応じて異なる特徴を持ち、セキュリティや利便性、コストの面で異なるバランスを提供している。

知識認証は、ユーザが知っている情報をもとに認証を行う方式であり、最も一般的な認証方法である。代表的な例はパスワード認証であり、ユーザ ID とそれに対応するパスワードを組み合わせることによって、本人確認を行う。この方法は簡便で広く普及しているが、パスワードの使い回しや盗難、推測攻撃によりセキュリティリスクが伴う。また、ユーザがパスワードを覚えておく必要があるため、利便性に関しては多少の負担がかかる。

所有物認証は、ユーザが物理的に所有しているアイテムを用いて認証を行う方式である。代表的なものとしては、鍵やキャッシュカード、ワンタイムパスワードを発行するトークンなどが挙げられる。所有物認証は、物理的なアイテムが盗まれた場合に不正アクセスを招くリスクがあるため、安全性の面で知識認証に劣ることがある。しかし、所有物がユーザ自身に帰属している限り、

パスワード認証に比べて不正アクセスのリスクは比較的低くなる。

生体認証は、ユーザの生物学的特徴を利用して認証を行う方法であり、指紋認証、顔認証、虹彩認証、静脈認証などが代表例として挙げられる。この方法の大きな利点は、ユーザ固有の生体情報をもとにして認証が行われるため、他人による不正ななりすましが非常に困難である点である。特に、静脈認証や虹彩認証は高いセキュリティ性を誇り、データセンターなどの高リスクな施設では広く活用されている。しかし、生体認証には専用の設備が必要となり、そのため導入コストが高くなる点が欠点である。また、利用者によっては生体情報が誤認識される場合もあるため、実用上の課題も存在する。

これらの認証方式は、セキュリティと利便性、そしてコストとのトレードオフの中で選択されるべきである。パスワード認証は低コストで実装可能であり、広く使用されているが、セキュリティの面では不安が残る。生体認証や所有物認証は高い安全性を提供するものの、コストや設備の整備が求められる。近年では、これらの認証方式を組み合わせた「多要素認証」が広まりつつある。例えば、パスワードに加えてワンタイムパスワードを要求する方式や、銀行のキャッシュカードに暗証番号を組み合わせる方式がその例である。多要素認証は、複数の異なる認証手段を組み合わせることで、セキュリティの強化を図る方法であり、オンラインバンキングやウェブサービスでの安全性向上に貢献している。

2.1.3 パスワード認証

パスワード認証は知識認証の一形態であり、サービスにアクセスするための認証手段として広く利用されている。この方法では、ユーザが自分の ID と事前に設定したパスワードを用いて認証を行う。具体的には、ユーザ U がサービス S に対して、以下の情報を送信する。

$$U \rightarrow S : \text{ID}(S), \text{password}(S)$$

ここで、 $\text{ID}(S)$ はユーザの識別子、 $\text{password}(S)$ はユーザ U がサービス S に設定したパスワードを示す。認証の際、パスワードは事前にサービス S とユーザ U の間で共有されていることが前提となっている。通信は通常、SSL (Secure Sockets Layer) や TLS (Transport Layer Security) などの暗号化通信プロトコルを利用して保護される。これにより、送受信されるパスワード情報の機密性が確保される。

この方式の利点は、ユーザが覚えやすいパスワードを使用することで簡単に認証を行える点である。しかし、いくつかの欠点も存在する。最も重大な問題の一つは、通信経路のセキュリティが破られた場合に発生するリプレイ攻撃である。リプレイ攻撃とは、認証に使用された通信を第三者が傍受し、後にそれを再送信して不正に認証を通す攻撃方法である。もし、SSL 暗号化の脆弱性が突かれた場合、攻撃者がパスワード情報を盗み、同じ認証情報を用いてサービスにアクセスすることが可能になってしまう。

このような理由から、パスワード認証方式は依然として広く使用されているものの、そのセキュリティ強化が求められる状況にある。対策としては、より強力な暗号化技術や、多要素認証 (MFA) の導入が有効であると考えられている。

2.1.4 チャレンジレスポンス認証

チャレンジレスポンス認証は、ユーザとシステム間での安全な認証を実現するための手法の一つである。この方式では、システムがユーザに対して「チャレンジ」と呼ばれる情報（例：ランダムなデータ）を提示し、ユーザはあらかじめ設定された秘密の情報をもとに計算した結果を「レスポンス」として返答することで認証が行われる。重要な点は、認証の際にシステムとユーザ間で直接的な認証情報のやり取りが行われないことであり、代わりにユーザはその情報を知っていることを証明する計算を実行する。

具体的には、システムはランダムなデータをユーザに送信し、ユーザはそのデータに基づいて秘密情報を用いて計算を行い、その結果をシステムに返送する。この際、システムは受け取ったレスポンスが自ら生成したチャレンジに適合しているかを検証し、一致すれば認証が成功する仕組みである。これにより、ユーザは秘密情報を直接的に送信することなく、その情報を所持していることを証明できる。

チャレンジレスポンス認証の最大の利点の一つは、リプレイ攻撃に対する耐性を持つことである。リプレイ攻撃とは、過去の認証情報を悪用して不正アクセスを試みる手法であるが、チャレンジレスポンス認証では毎回異なるチャレンジが使用されるため、過去のレスポンスを再利用することができない。このような特性により、チャレンジレスポンス認証は、既存の認証方式に比べてセキュリティが高いとされている。

さらに、この方式は、ユーザの認証情報を漏洩から守りつつも、高い利便性を提供することが可能である。例えば、パスワードや PIN コードのように、実際に認証情報を送信する必要がないため、ネットワーク上で情報が漏洩するリスクを低減できる。このような利点から、チャレンジレスポンス認証は様々なセキュリティが求められるシステムにおいて有効な認証方法として利用されている。

2.2 機械学習

2.2.1 機械学習とは

機械学習とは、データからパターンを学び、予測や分類などのタスクを実行するための手法である [7][8]。事前に与えられたデータをコンピュータが学習し、未知のデータに対しても適切な処理を行うことができる。

機械学習には、データをもとに予測や分類を行うための、モデルと呼ばれる数学的な構造を使用する。モデルは、与えられたデータに基づいて予測を行うために、事前に「学習」と呼ばれるプロセスを経る。この学習過程では、モデルの出力と実際の正解との差を表す損失関数を用いてモデルの精度を評価し、その誤差を最小化するようにモデルのパラメータ（重み）が調整される。また、学習アルゴリズムには誤差逆伝播法と呼ばれる手法が用いられ、モデルのパラメータはこの方法を通じて最適化される。一般的な最適化手法としては、確率的勾配降下法（SGD）、Adam、AdaGrad などが使用され、これらは効率的に学習を進めるための手段となる。

機械学習の学習過程は次のように進行する。まず、問題を解決するためのモデルが選択され、そのモデルに対して大量の学習データ（説明変数と正解データのペア）が提供される。次に、モデルに対してランダムに初期値が設定された重みが与えられ、学習データがモデルに入力される。その後、損失関数によりモデルの予測誤差が評価され、この誤差を最小化するために誤差逆伝播法を用いて重みが更新される。このプロセスは繰り返し行われ、最終的に最適なモデルが得られる。学習が完了した後は、学習データとは異なる新たなデータ（テストデータ）を用いて、モデルがどれだけ汎用性を持っているかを評価する。この評価では、モデルが学習データに過度に適応しており、未知のデータに対する汎化性能を獲得していない可能性がある。この現象を過学習と呼ぶ[7]。過学習を起こしてしまうと、モデルは未知のデータに対応できない状態となるため、これを避ける必要がある。

機械学習では、入力や出力は通常ベクトルや行列といったテンソル形式で扱われ、データの処理を効率的に行うための構造として層ごとに異なる処理が行われる。最初の層は入力層、最後の層は出力層と呼ばれ、それらの間に位置する層は中間層（隠れ層）と呼ばれる[7]。これらの層を通じて、機械学習モデルは与えられた問題に対する解を導き出す。

2.2.2 様々な機械学習モデル

機械学習の領域において広く使用されているモデルの一つに、ニューラルネットワークがある。本研究では、モデルとして畳み込みニューラルネットワークを用いるため、ここではニューラルネットワークを中心に機械学習モデルの紹介を行う。

ニューラルネットワークとは、入力データを処理して出力を生成するための数理モデルである[7]。層状のノード（ニューロン）で構成されており、ノード間の接続に対する重みやバイアスを調整することによって学習を行う。学習手法として、誤差逆伝播法や最適化アルゴリズムを使用してモデルを訓練する。

他にも代表的な機械学習モデルとして、多層パーセプトロン（MLP）、時間的な依存関係を処理する再帰型ニューラルネットワーク（RNN）、画像データの処理に特化した畳み込みニューラルネットワーク（CNN）、そして自然言語処理で注目を集めた Transformer などが挙げられる[8]。これらモデルは機械学習分野の様々な領域において活用されているが、機械学習モデルは自在に組み合わせることが可能であり、例えば RNN で使われる層と CNN で使われる層の両方を取り入れ

た機械学習モデルも存在する。

2.2.3 埋め込み層

埋め込み層は、自然言語処理の分野において単語を効率的に扱うために開発された技術である[8]。この層は、単語の意味を高次元のベクトル空間にマッピングすることで、コンピュータが単語の意味を理解できるようにする。単語の分散表現と呼ばれる手法をもとにしており、各単語は数値ベクトルとして表現され、ベクトル間の距離や内積を計算することで、単語間の類似性や関係性を捉えることができる[8]。

埋め込み層は、入力データ（通常は単語の ID）を受け取り、その ID に対応するベクトルを出力する。具体的には、入力は One-Hot ベクトルとして表現され、これが埋め込み行列との掛け算によって対応する単語ベクトルに変換される。埋め込み層の特徴的な点は、近い意味を持つ単語の ID が類似したベクトル値を持つように学習することによって、言葉の意味が数値的に反映される点である。

この技術は、特に自然言語処理のタスクにおいて広く用いられており、例えば文脈に基づく単語の意味を理解するための基盤となる。埋め込み層を利用した応用例としては、ベクトル空間内での単語類似度を計算することによって、意味的に関連する単語を検索するベクトル類似性検索が挙げられる。これにより、従来の辞書的なアプローチを超えた意味の深い情報処理が可能となった。

2.2.4 長短期記憶 (LSTM)

長短期記憶 (LSTM) とは、時系列データを効果的に扱うために設計されたニューラルネットワークの一種である[11]。LSTM は、入力された時系列データを順次処理し、過去の情報を保持しながら予測を行う能力を持つ。この特徴により、特に時間的依存性を考慮したデータに対する推論が得意であり、長期的な依存関係を捉えることが可能である。

LSTM の主な特徴は、内部に「状態」を保持する点にある。この状態は、ネットワークが過去の情報をどの程度保持するかを制御し、過去のデータから学習した情報を現在の予測に活かす役割を果たす。一般的な再帰型ニューラルネットワーク (RNN) は過去の情報を保持するのが苦手であり、長期間の依存関係を捉えるのが難しいが、LSTM は内部にゲート機構を持ち、重要な情報を忘れずに保持することができる[11][12]。

LSTM では、各時刻における入力データをもとに状態を更新し、次の時刻の予測を出力する。この過程で、LSTM は「忘却ゲート」や「入力ゲート」、「出力ゲート」といった特殊なゲートを用いて、どの情報を保持し、どの情報を忘れるべきかを調整する。この仕組みによって、長期間にわたる依存関係を効率的に学習することが可能となる。

さらに、LSTM は単方向に加えて、双方向 LSTM (Bi-LSTM) という手法を用いることができる。既存研究で使用されている双方向 LSTM は、時系列データを前方向と後方向の両方で学習・推論を行うことができ、これにより動画解析や音声認識などのタスクにおいて、より多角的な理解が得られる。

総じて、LSTM は時系列データに対して強力な予測能力を発揮し、自然言語処理や音声処理、動画解析など、時間的依存性を持つ多くの応用領域で活用されている。

2.2.5 畳み込みニューラルネットワーク (CNN)

畳み込みニューラルネットワーク (CNN) は、特に画像データや空間的な構造を持つデータの解析に適したニューラルネットワークの一種である[13][14]。CNN は、入力されたデータに対して「畳み込み」という演算を行うことで、特徴を自動的に抽出し、効果的にデータを処理することができる。これにより、従来の全結合型ニューラルネットワークと比べて、画像認識や物体検出、顔認識などのタスクにおいて優れたパフォーマンスを発揮する。

CNN は、主に 3 つの層から構成される。1 つ目は「畳み込み層」であり、ここでは入力画像に対して複数のフィルター（カーネル）を適用し、画像内の特徴を抽出する。このフィルターは、エッジやテクスチャ、形状などの低次元の特徴を捉えることができ、これにより画像内の局所的なパターンを学習することが可能になる。畳み込み演算によって得られた特徴マップは、画像の空間的な構造を保持しながら、重要な情報を強調する。

次に、畳み込み層で得られた特徴を縮小・集約する「プーリング層」が続く。プーリング層では、特徴マップから代表的な値を選択（例えば最大値を取る最大プーリング）することで、計算量を削減し、特徴量の次元を圧縮する。また、この過程で画像内の細かな位置に依存せず、より抽象的な特徴を捉えることができ、モデルがよりロバストになる。

最後に、CNN の出力層として「全結合層」があり、ここでは抽出された特徴をもとに最終的な分類や回帰の予測が行われる。この全結合層は、最終的な結果を出力するために使用され、CNN が学習した特徴をもとに最終的な推論を行う。

CNN は、画像認識だけでなく、音声認識や自然言語処理など、空間的または時系列的な構造を持つデータにも広く応用されている。また、畳み込み層が自動的に特徴を抽出するため、事前に特徴量を手作業で設計する必要がなく、深層学習における強力なモデルとして非常に高い汎用性を誇る。

総じて、CNN は画像処理の分野において革命的な手法であり、物体認識、顔認識、医療画像解析など、数多くの実世界のアプリケーションで成功を収めている。

第 3 章

既存研究

3.1 人間計算可能なパスワード

3.1.1 人間計算可能なパスワードとは

人間計算可能なパスワードとは、2016 年に Manuel Blum らによって提案された認証方式であり、コンピュータを用いることなく、ユーザが自身の脳内で計算を完結させるチャレンジレスポンス認証の一種である。この方式の特徴は、記憶と計算能力を組み合わせることで認証を行う点にあり、知識認証の一形態として分類される。そのため、「人間計算可能」という名称が用いられている[1]。

この認証方式では、パスワードを生成・記憶する段階と、実際に認証を行う段階の二段階に分けられる。前者は「PRPE」と呼ばれ、パスワードの生成と記憶のプロセスが含まれる。後者は「PROC」と呼ばれ、ユーザが秘密鍵とチャレンジ（入力）に対して特定の計算を実行し、認証応答を得る段階に該当する[1][2]。

さらに、PRPE は「MEM」と「COM」の二つの要素に細分化される。MEM はユーザが秘密鍵を記憶し、必要に応じてリハーサルを行い、記憶を強化するプロセスである。一方、COM は計算スキームの学習を指し、ユーザがどのような計算を行うかを習得することが含まれる。これにより、認証時には記憶した鍵と学習した計算手順に基づいて応答を生成し、正しく認証が行われる。

3.1.2 人間計算可能な定義

人間計算可能であることを定義するために、まず人間の計算能力を一種の計算機としてモデル化する[2]。このモデルでは、人間をチューリングマシンに類似した存在とみなしつつ、記憶の特性を考慮して以下のような仮定を導入する。

I 人間の計算モデル

- (i) 人間はチューリングマシンと同様に計算を実行するが、記憶の仕組みが異なる。具体的には、長期記憶と短期記憶の二つの記憶領域を持つ。
- (ii) 長期記憶の容量は理論的には無限大とする。ただし、新しい情報を書き込むには一定の時間を要する。一方で、すでに記憶した情報の読み出しは即座に行える。
- (iii) 短期記憶は、情報の読み書きが極めて高速で行われるが、容量には大きな制約があり、通常

2～3 個の情報しか保持できない。

II 人間計算可能であるための条件

上記の記憶モデルを前提とし、人間が計算を行うプロセス（PRPE および PROC）が人間計算可能であるための具体的な条件を以下のように定義する。

（i）スキーマ学習の時間制約（COM）：

計算の手順を学習し、実際に試行するリハーサルを含めた学習時間は 10 分以内であること。

（ii）準備段階の時間制約（PRPE）：

スキーマの学習（COM）と秘密鍵の記憶（MEM）を合わせた準備にかかる時間は 2 時間以内で完了すること。

（iii）認証時の計算時間（PROC）：

実際に認証を行う際、与えられた入力と秘密鍵を用いて応答を計算する時間は 1 分以内であること。

（iv）スキーマの制約：

ユーザが記憶するスキーマの数は最大 3 つとし、短期記憶の限界を考慮して可能な限り 2 つのチャンクを記憶のポインタとして活用することが望ましい。

（v）リハーサルの実施可能性：

ユーザは、スキーマや秘密鍵を記憶した後、自らリハーサルを行い、正しく記憶されているかを確認できること。

これらの条件は、パスワード認証が完全に脳内で完結することを保証するために設定されている。もし計算に外部の記憶媒体（PC や紙など）を必要とする場合、認証の秘密情報が外部に漏洩する可能性が生じる。また、計算に要する時間が過度に長いと実用性が失われ、現実的な認証方式としての適用が困難になる。したがって、人間計算可能なパスワードの設計においては、計算負担とセキュリティのバランスを考慮し、これらの制約を満たすことが求められる。

3.1.3 一桁チャレンジ

一桁チャレンジとは、人間計算可能なパスワード認証プロセスにおいて、問題（チャレンジ）と秘密鍵に基づいて計算を行い、一桁（0～9）の数値を応答（レスポンス）として生成する方式である。この手法の特徴は、画像と数値の対応関係を利用する点にある。ユーザは事前に特定の画像と対応する数値を記憶し、それらを用いて認証時に計算を行うことで、外部デバイスを使用せずに安全な認証を実現する。

一桁チャレンジは、準備フェーズと認証フェーズの二段階に分けられる。

I 準備フェーズ

一桁チャレンジでは、ユーザは特定の画像を見た際に、それに紐づけられた数値を思い出せる必要がある。はじめに、ユーザはあらかじめ N 枚の画像とそれに対応する数値の組み合わせ、および計算に用いる関数 f を記憶する。前者の対応関係をマッピング関数 σ と定義する。マッピング関数 σ は、画像を入力として受け取り、 $0 \sim 9$ の整数を出力する関数である。これに加え、ユーザは認証時に用いる演算を行うための関数 f も記憶する。この関数 f は、複数の入力値を組み合わせ一桁の数値を計算するものである。

II 認証フェーズ

認証時には、ユーザが事前に登録した最大 100 枚の画像の中からランダムに 14 枚の画像が提示される。ユーザは提示された画像をもとに、それぞれ対応する数値を思い出し、定義された計算式に従って計算を行う。この計算により得られた一桁の値が、認証に使用される一桁の認証情報となる。ここでは、以下の数式によって認証情報が生成されるものとする。

$$f(x_0, x_1, \dots, x_{13}) = x_j + x_{12} + x_{13} \bmod 10$$

ここで、 j は次の式で定義される。

$$j = x_{10} + x_{11} \bmod 10$$

この方式では、認証に使用する情報はユーザの記憶内にのみ存在し、外部の記録媒体に依存しない。そのため、秘密鍵がデバイスや紙に保存されることによる漏洩リスクがなく、高いセキュリティを確保できる。また、計算がシンプルであるため、ユーザの負担を抑えながら安全な認証を実現できる点が特徴である。

画像番号	0	1	2	3		10	11	12	13
画像 I					...				
$\sigma(I)$	1	2	8	6		4	9	7	3

$$j = \sigma(\text{tennis racket and balls}) + \sigma(\text{bird}) \bmod 10 = 3$$

$$f(x_0, x_1, \dots, x_{13}) = \sigma(\text{blue car}) + \sigma(\text{stack of books}) + \sigma(\text{coffee cup}) \bmod 10 = 6$$

図 3.1.3-1 一桁チャレンジにおいて記憶するマッピング関数 σ とその計算例

3.1.4 人間計算可能関数

前節で登場した関数 f について、既存研究では、これがより一般的な形で導入されている[1]。具体的には、ほとんどの人間が桁の算術演算に慣れていることを考慮し、すべての候補関数 $f: Z_k^d \rightarrow Z_d$ に対して $d = 10$ に固定した関数族を導入している。

整数 $k_1 > 0$ および $k_2 > 0$ を与えたとき、関数 $f_{k_1, k_2}: Z_{10+k_1+k_2}^{10}$ を以下のように定義する。

$$f_{k_1, k_2}(x_0, \dots, x_{9+k_1+k_2}) = x_j + \sum_{i=10+k_1}^{9+k_1+k_2} x_i \bmod 10$$

ここで、 j は次の式で定義される。

$$j = \sum_{i=10}^{9+k_1} x_i \bmod 10$$

既存研究ではさらに、人間計算可能関数の候補群について、その使用性とセキュリティの両面から詳細な分析が行われている[1]。

使用性の観点においては、関数の計算に要する時間、および関数 σ の記憶／保持の容易さが評価されている。特に、関数 f_{k_1, k_2} の計算時間は加算操作および記憶操作の回数に基づいて分析され、人間が一定の訓練を受けることで実用的な速度で計算可能であることが示唆されている。また、関数 σ の記憶に関しては、記憶補助ツールの活用や継続的な使用による自然な記憶強化の可能性が指摘されている。

セキュリティの観点においては、関数の安全性はパラメータ k_1 および k_2 に依存することが示されており、適切な設定を行うことで攻撃者がパスワードを推測する確率を制御できることが明らかにされている。また、パスワードの暗号学的ハッシュ化を前提とすることで、サーバ侵害時のオフライン攻撃への耐性も確保できることが示されている。

以上のことから、既存研究では、この方式は適切なパラメータ設定のもとで計算可能性とセキュリティを両立できる可能性があると結論づけられている[1]。

3.1.5 認証プロセス

認証プロセスは、ユーザが記憶したマッピング関数 σ および関数 f をもとに、チャレンジレスポンス方式を用いて行われる。はじめに、ユーザのローカルコンピュータ上で関数 σ が秘密裏に生成される。このとき、必要に応じて記憶補助ツールが提供されるが、このツールは記憶後に破棄される。具体的には、アカウント A_i (パスワード pw_i) の認証プロセスは、以下の手順で行われる。

1. ローカルコンピュータがランダムに λ 個の一桁チャレンジ $c_{i1}, \dots, c_{i\lambda} \in X^\lambda$ を生成する。

2. ユーザは記憶した関数 σ を用いて、それぞれのチャレンジに対応する値を取得し、関数 f を適用することでパスワード $pw_i = (f(\sigma(C_{i1})), \dots, f(\sigma(C_{i\alpha})))$ を計算する。
3. 認証サーバは、計算されたパスワード pw_i の暗号的ハッシュ値を保存する。
4. 生成されたチャレンジ $C_{i1}, \dots, C_{i\alpha}$ は、ユーザのローカルコンピュータ上の公開領域に保存され、攻撃者が閲覧できる可能性がある。
5. 認証時には、ユーザが公開されたチャレンジを取得し、記憶した関数 σ を用いてパスワード pw_i を再計算する。
6. 認証サーバは、ユーザが入力した pw_i のハッシュ値と保存されたハッシュ値を照合し、一致すれば認証が成功する。

この方式により、ユーザの秘密情報を直接通信することなく、記憶に基づく認証を実現することが可能となる。

3.2 埋め込み層および双方向長短期記憶による安全性評価

既存研究では、認証情報が漏洩した際における攻撃手法の一つとして、埋め込み層を前処理として導入した双方向長短期記憶 (Bi-LSTM) を用いた解析が実施されている [6] (表 3.2-1)。この研究では、攻撃者が通信経路などを介してユーザの認証情報の一部を取得した状況を想定し、機械学習を用いたなりすましの可能性を検証している。具体的には、ユーザが認証時に閲覧した 14 枚の画像と、それに基づいて入力した一桁の数値が漏洩したケースを考える。モデルの学習においては、これらの 14 枚の画像を説明変数、ユーザが実際に入力した数値を目的変数として設定し、ニューラルネットワークによる予測の精度を評価する。また先行研究では、 $(k_1, k_2) = (2, 2)$ 型の人間計算可能関数のみを対象として分析が行われている。具体的には、この関数は

$$f_{2,2}(x_0, \dots, x_{13}) = x_j + x_{12} + x_{13} \bmod 10, \quad j = x_{10} + x_{11} \bmod 10$$

と定義され、14 個の入力値から適切なレスポンスを算出する方式が採用されている。

学習済みモデルがユーザの入力値を高い精度で予測できる場合、攻撃者が同様の手法を用いてなりすましを行う危険性が示唆される。一方で、予測精度が低い場合には、人間計算可能なパスワードが機械学習に対して十分な耐性を持つ可能性が示される。研究の目的は、この攻撃シナリオにおいて Bi-LSTM を用いた解析がどの程度成功するかを詳細に検証することであった。

表 3.2-2 は、この研究の実験結果である。結果として、ユーザが記憶すべき画像枚数が 26 枚の場合、予測精度は最大で 55.81% に達することが確認されている。一方、画像枚数が 50 枚および 100 枚の場合の予測精度は最大で 17.20% にとどまり、ユーザが記憶する画像枚数が増加するにつれて予測が困難になることが示されている。

層	層の種類	ノード数	出力形式
---	------	------	------

1	入力層	14	[14]
2	埋め込み層	42	[14, 3]
4	Bi-LSTM	56	[56]
5	Bi-LSTM	56	[56]
6	Bi-LSTM	56	[56]
7	全結合層	50	[50]
8	全結合層	50	[50]
9	全結合層	50	[50]
10	出力層	10	[10]

表 3.2-1 埋め込み層および Bi-LSTM の構成

データ数\画像枚数	N=26	N=50	N=100
1000	0.1020	0.1720	0.1420
5000	0.1624	0.1051	0.1002
10000	0.5581	0.1023	0.1121
50000	0.3302	0.1109	0.1021

表 3.2-2 埋め込み層および Bi-LSTM を用いた予測精度

第 4 章

研究手法

4.1 本研究の研究内容

本研究では、既存研究において比較的高い予測精度を示した手法を再現し、その結果を踏まえて新たなモデルを設計／評価する。具体的には、埋め込みを前処理とした双方向長短期記憶（Bi-LSTM）を用いた機械学習モデルを再現し、その予測精度を分析する [6]。さらに、従来手法の課題を考慮し、埋め込みを前処理とした畳み込みニューラルネットワーク（CNN）を導入することで、異なるアーキテクチャによる予測精度の向上の可能性を検討する。特に、 $(k_1, k_2) = (2, 2)$ 型および $(k_1, k_2) = (1, 3)$ 型の人間計算可能関数を対象とした予測精度の向上を目指し、異なるパラメータ設定におけるモデルの性能を比較する。具体的には、それぞれ次の式によって定義される人間計算可能関数を対象としている。

$$(k_1, k_2) = (2, 2) \text{ 型: } f_{2,2}(x_0, \dots, x_{13}) = x_j + x_{12} + x_{13} \bmod 10, \quad j = x_{10} + x_{11} \bmod 10$$

$$(k_1, k_2) = (1, 3) \text{ 型: } f_{1,3}(x_0, \dots, x_{13}) = x_j + x_{11} + x_{12} + x_{13} \bmod 10, \quad j = x_{10} \bmod 10$$

加えて、パラメータ k_1 および k_2 の値が、パスワードの安全性に与える影響を観察する。これにより、各パラメータ設定がどのように予測精度やセキュリティのバランスに関与するのかを明らかにし、より安全なパスワード設計のための指針を提供することを目的とする。

4.2 実験手法

4.2.1 タスク定義とモデルの設計

本研究では、生成したデータセットを訓練データと検証データに分割し、訓練データを用いてモデルの学習を行う。入力データは、一桁チャレンジにおいて提示される 14 枚の画像番号であり、出力データは、それに対応するレスポンス（0～9 の整数値）である。このタスクは、出力が 10 種類の離散的な値をとるため、10 クラス分類問題として定式化される。そのため、モデルの出力層には 10 個のニューロンを配置し、それぞれのニューロンが特定のレスポンスを表現する形で分類を行う。

なお、本研究における予測タスクは、合成関数 $f \circ \sigma$ の出力を推定することに相当する。すなわち、与えられた一桁チャレンジに対して、対応するレスポンスをどの程度の精度で再現できるか

を検証することが本実験の主題である。

4.2.2 実験に使用するデータセット

本研究の実験では、一桁チャレンジにおいて提示される 14 枚の画像番号と、それに対応するレスポンスを含むデータセットを使用する。具体的には、各データサンプルは、認証時に提示される 14 個の画像番号 (X0~X13) を特徴量とし、それに基づいて計算された 1 桁のレスポンス (Z) を正解ラベルとして構成される (図 4.2.2-1)。

一桁チャレンジにおいて提示される14枚の画像番号														レスポンス
X0	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	Z
34	66	34	50	30	83	5	74	11	2	80	44	84	2	6
19	52	89	65	83	58	3	89	88	47	32	29	14	42	0
38	83	35	50	0	72	71	99	14	94	25	12	6	31	3
							⋮							

図 4.2.2-1 実験に使用するデータセットの例 (N=100 の場合)

4.2.3 実験に使用する機械学習モデル

本研究では、埋め込み層を前処理として導入した畳み込みニューラルネットワーク (CNN) を用いて実験を行う (表 4.2.3-1)。既存研究では、埋め込み層と双方向長短期記憶 (Bi-LSTM) を組み合わせた手法によって比較的高い予測精度が得られており、特に埋め込み層がマッピング関数 σ の近似的な表現として機能している可能性が指摘されていた[6]。しかし、Bi-LSTM は長期依存関係を考慮するために計算コストが増大しやすく、ハイパーパラメータの調整が難しいという課題があった。

一方、本研究において予測対象とする関数 f は、 x_{10} , x_{11} によって決定される x_j と、 x_{12} , x_{13} の間に局所的な関係性が存在し、これらの要素が出力に大きく影響を与える。このような局所的な依存関係を適切に学習するには、特徴量間の相互作用を考慮できるモデルの選択が重要である。CNN は局所的な関係性を重点的に学習しつつ、ノイズの影響を抑えられるため、関数 f の構造に適していると考えられる。さらに、CNN は並列計算が可能であるため、学習の安定性向上やハイパーパラメータ調整の容易さが期待できる。

このような背景から、本研究では、埋め込み層と CNN を組み合わせたアーキテクチャを採用し、その有効性を検証する。

層	層の種類	ノード数	出力形式
1	入力層	14	[14]
2	埋め込み層	42	[14, 3]

4	畳み込み層	384	[12, 32]
5	畳み込み層	640	[10, 64]
6	平坦化層	640	[640]
7	全結合層	30	[30]
8	全結合層	30	[30]
9	全結合層	30	[30]
10	出力層	10	[10]

表 4.2.3-1 埋め込み層および CNN の構成

4.2.4 実験の手順

本研究の実験は、以下の手順で実施した。

1. データの生成：
 - ・乱数を用いて、一桁チャレンジに対応する秘密関数 σ をランダムに生成する。
 - ・関数 σ に基づき、チャレンジ-レスポンスの組み合わせをランダムに生成する。このとき生成したチャレンジ-レスポンスの組み合わせの総数は、漏洩したデータ数と同等と仮定する。
2. データの分割：
 - ・生成したチャレンジ-レスポンスの組み合わせを、8：2 の割合で訓練データと検証データに分割する。
3. モデルの学習：
 - ・訓練データを用いて機械学習モデルを学習させる。
4. 実験と評価：
 - ・検証データにおける正解率を観察し、その結果を人間計算可能なパスワードの予測精度として実験を行う。
 - ・本実験により、機械学習を用いたセキュリティ評価の有効性を検証する。

第 5 章

結果

5.1 埋め込み層および双方向長短期記憶を用いた実験結果

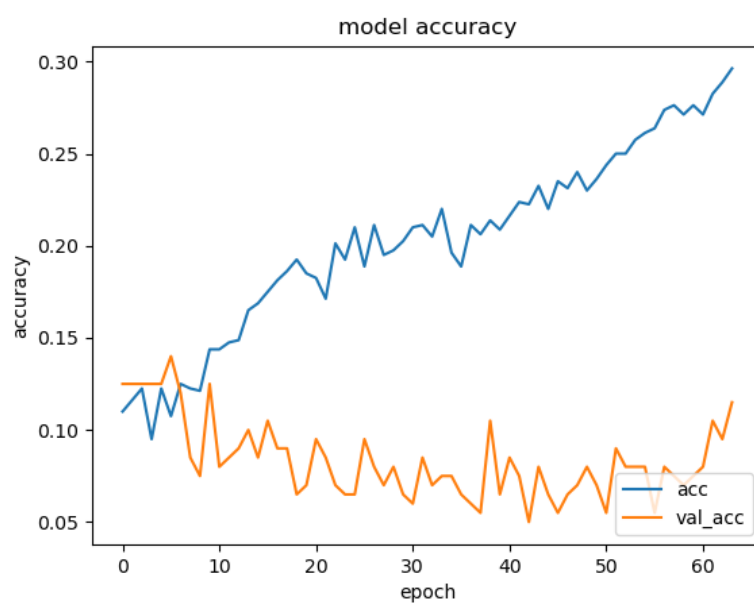


図 5.1-1 N=26, データ数 1000 における Bi-LSTM を用いた予測精度

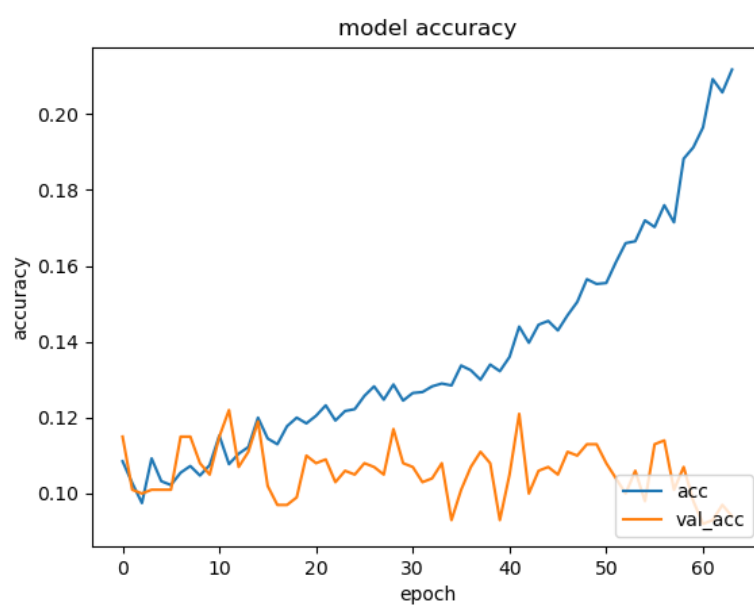


図 5.1-2 N=26, データ数 5000 における Bi-LSTM を用いた予測精度

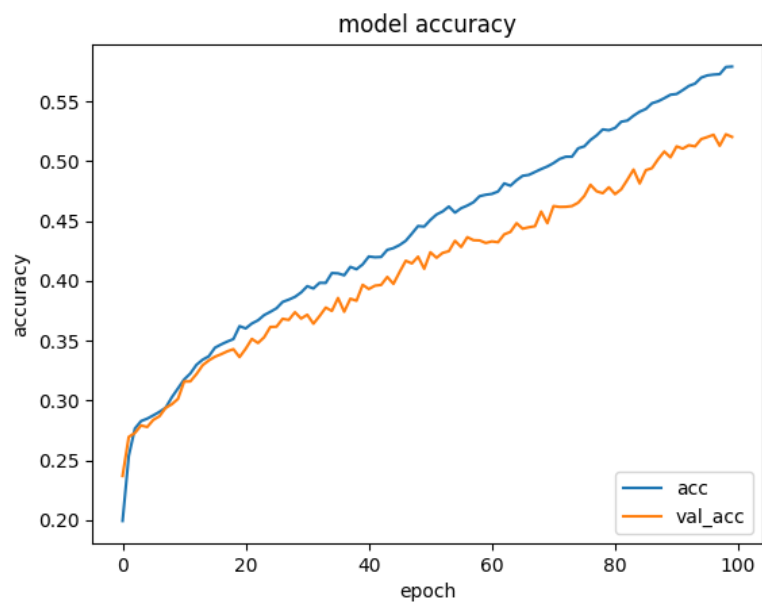


図 5.1-3 N=26, データ数 10000 における Bi-LSTM を用いた予測精度

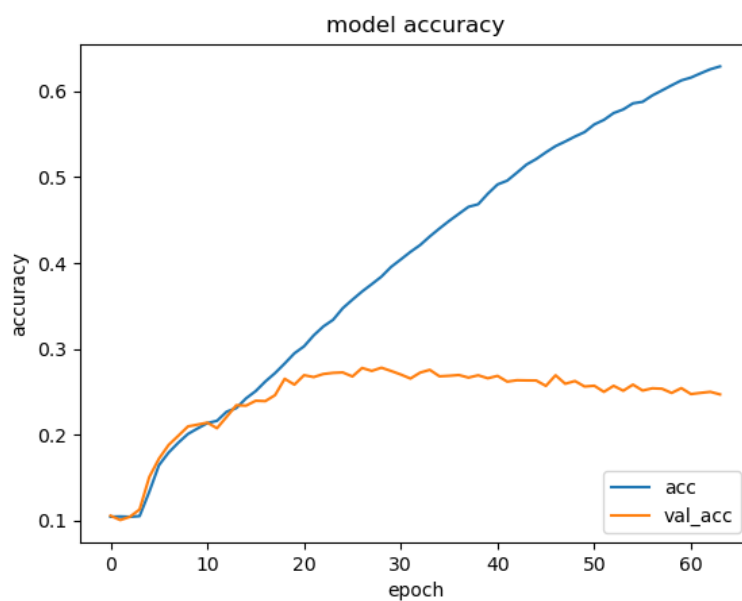


図 5.1-4 N=26, データ数 50000 における Bi-LSTM を用いた予測精度

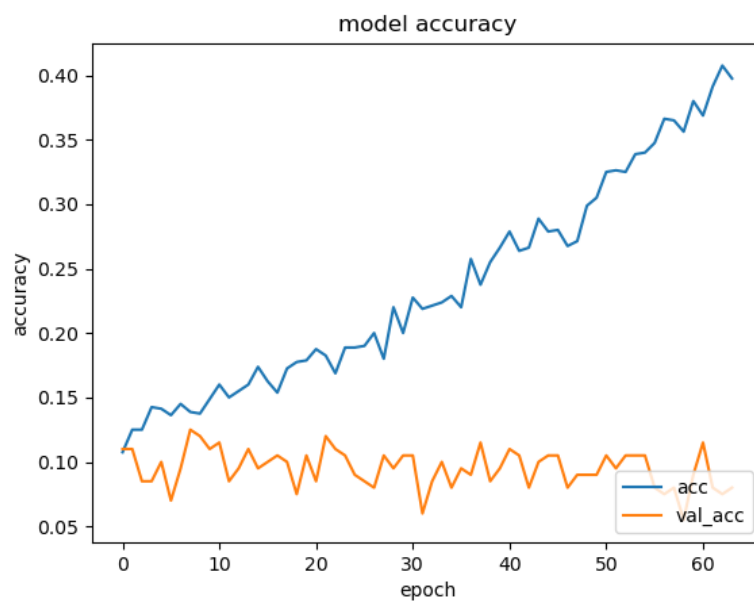


図 5.1-5 N=50, データ数 1000 における Bi-LSTM を用いた予測精度

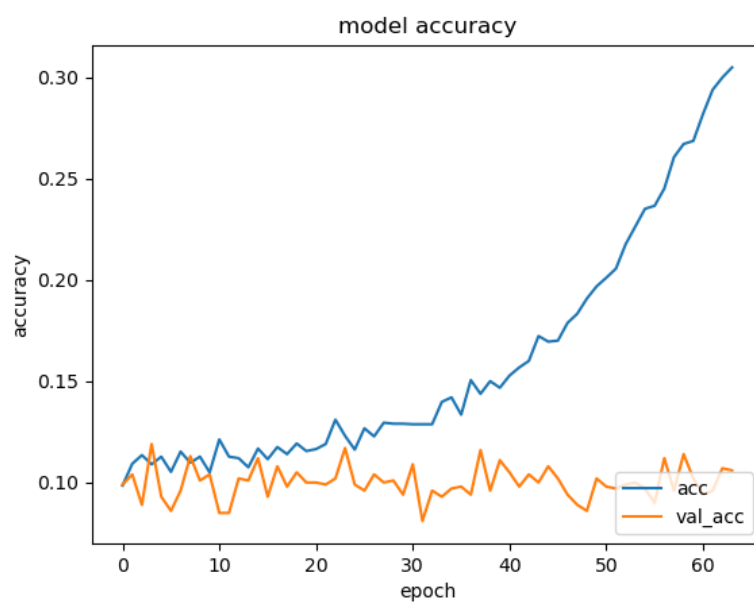


図 5.1-6 N=50, データ数 5000 における Bi-LSTM を用いた予測精度

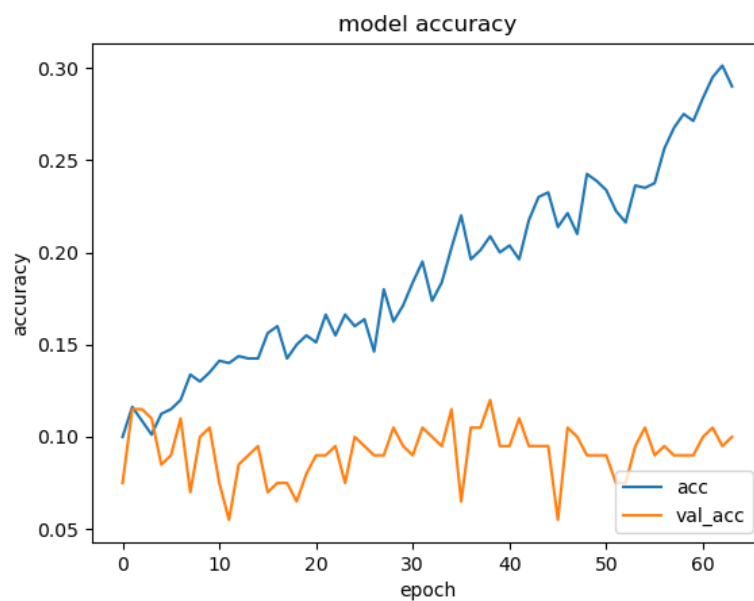


図 5.1-7 N=50, データ数 10000 における Bi-LSTM を用いた予測精度

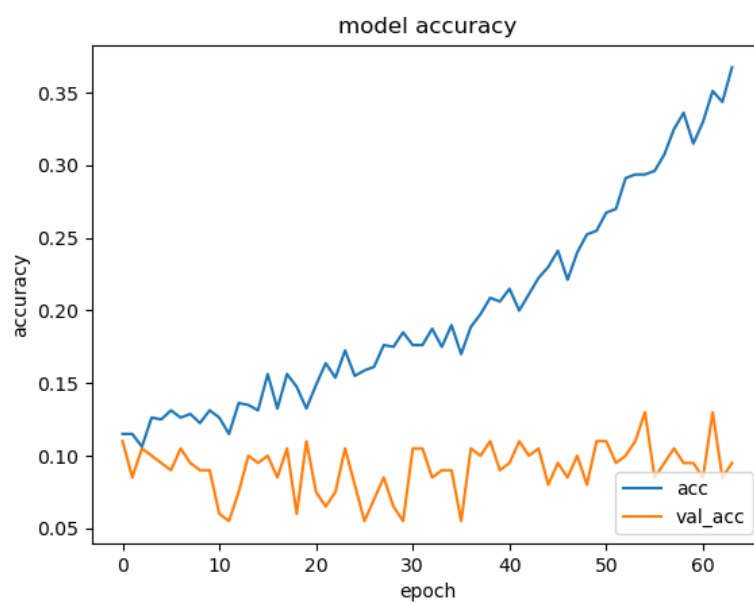


図 5.1-8 N=50, データ数 50000 における Bi-LSTM を用いた予測精度

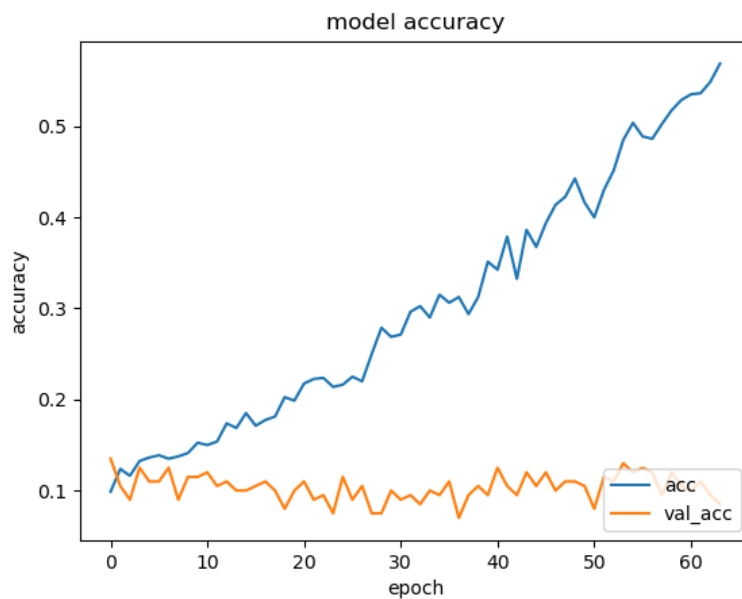


図 5.1-9 N=100, データ数 1000 における Bi-LSTM を用いた予測精度

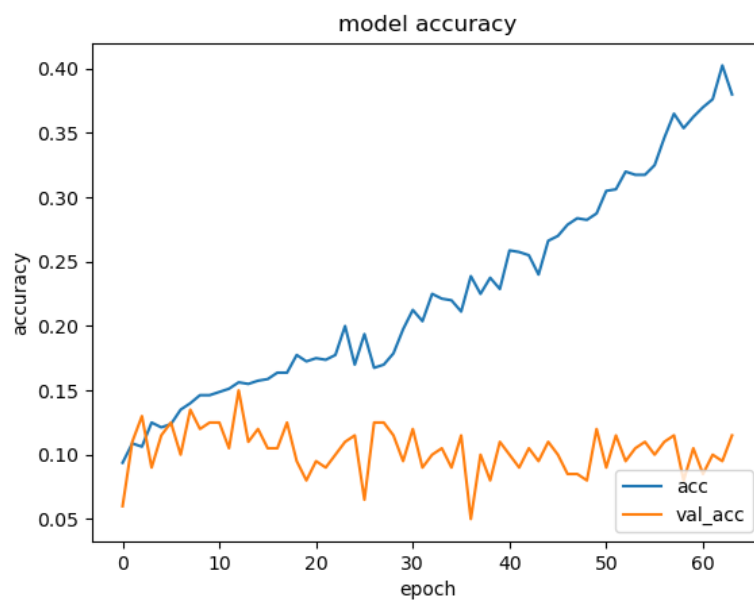


図 5.1-10 N=100, データ数 5000 における Bi-LSTM を用いた予測精度

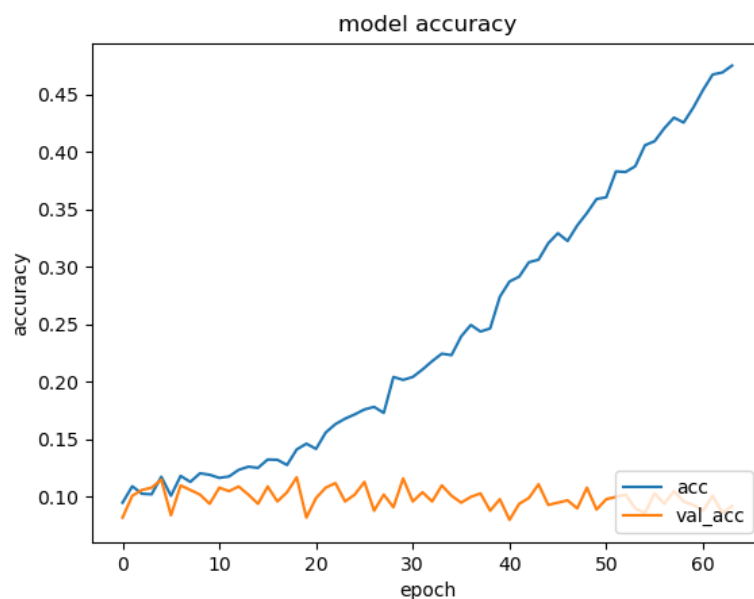


図 5.1-11 N=100, データ数 10000 における Bi-LSTM を用いた予測精度

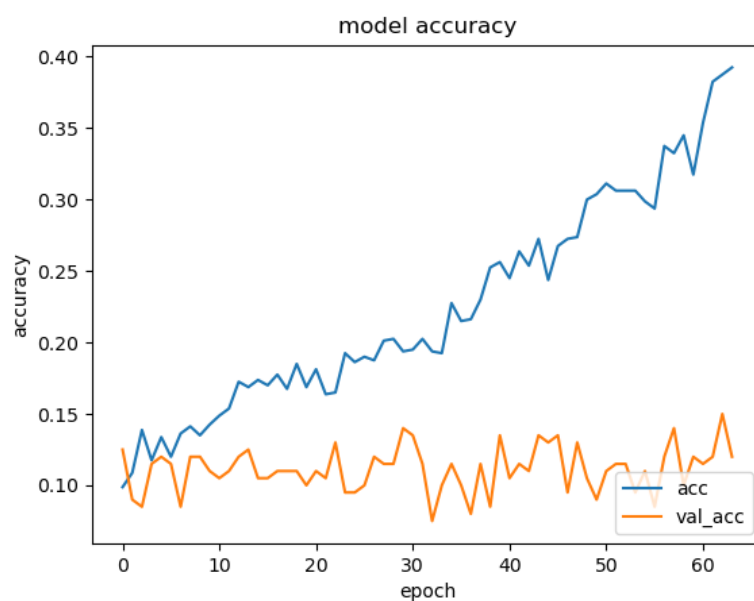


図 5.1-12 N=100, データ数 50000 における Bi-LSTM を用いた予測精度

5.2 埋め込み層および畳み込みニューラルネットワークを用いた実験結果

本節では、埋め込み層を前処理として導入した畳み込みニューラルネットワーク（CNN）を用いた実験を行った。特に、人間計算可能関数におけるパラメータ設定 $(k_1, k_2) = (2, 2)$ および

$(k_1, k_2) = (1, 3)$ の違いが、モデルの予測精度にどのような変化をもたらすのかを分析し、パスワードの安全性に与える影響を評価した。

5.2.1 $(k_1, k_2) = (2, 2)$ における実験結果

人間計算可能関数において、 $(k_1, k_2) = (2, 2)$ とした場合の予測精度を以下の表 5.2.1-1 および図 5.2.1-1～図 5.2.1-12 に示す。ユーザが記憶する画像枚数が 26 枚の場合、データ数 50000 において予測精度は最大で 60.84%に達した。一方で、その他の場合の予測精度は最大でも約 20%にとどまり、ランダムに予測した場合とほとんど変わらない結果となった。特に、ユーザが記憶する画像枚数を 50 枚および 100 枚に設定した場合、データ数が 50000 に達すると、訓練データに対する学習すら進まない現象が確認された。また、Bi-LSTM を用いた実験と同様に、ユーザが記憶する画像枚数が増加するにつれて、予測精度が低下する傾向が見られた。

データ数\画像枚数	N=26	N=50	N=100
1000	0.1603	0.1517	0.1650
5000	0.1160	0.1182	0.1314
10000	0.2115	0.1145	0.1128
50000	0.6084	0.1030	0.1038

表 5.2.1-1 $(k_1, k_2) = (2, 2)$ 型における埋め込み層および CNN を用いた予測精度

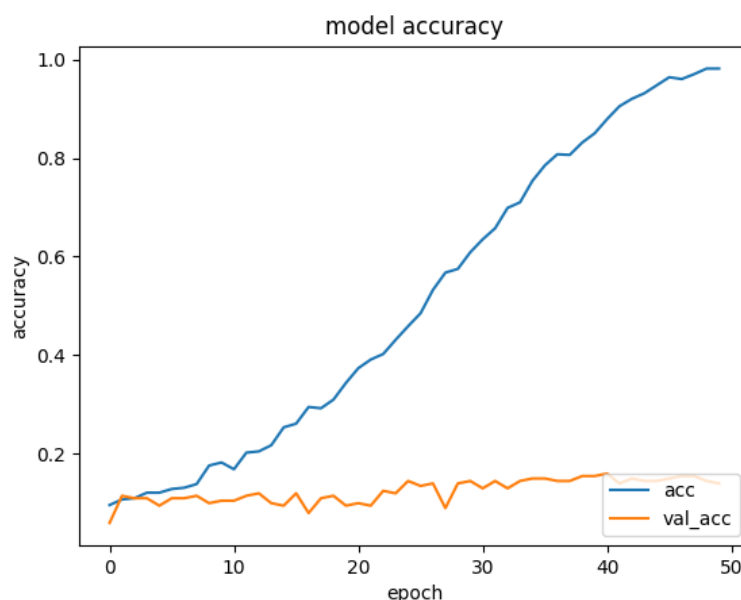


図 5.2.1-1 $(k_1, k_2) = (2, 2)$ 型 : N=26, データ数 1000 における CNN を用いた予測精度

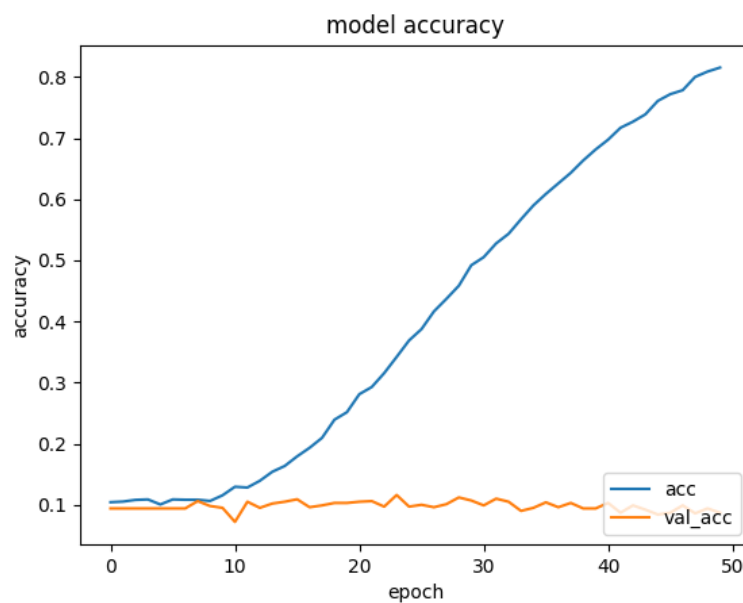


図 5.2.1-2 $(k_1, k_2) = (2, 2)$ 型 : $N=26$, データ数 5000 における CNN を用いた予測精度

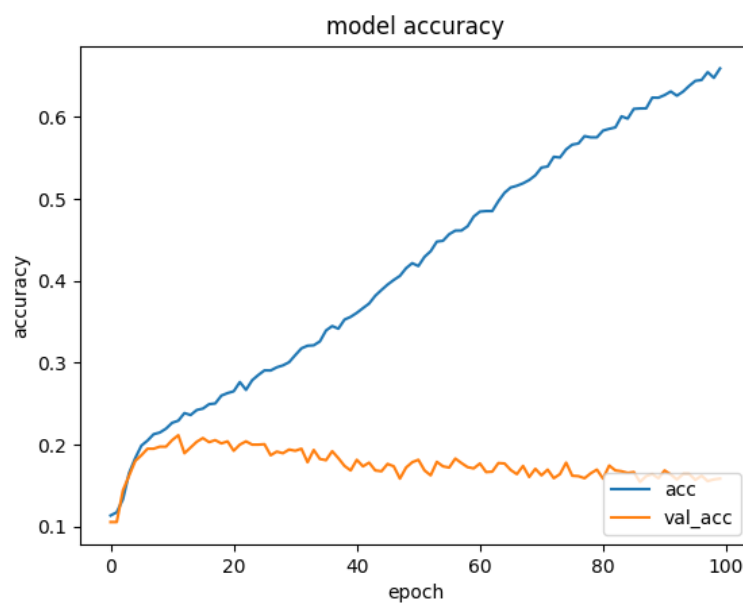


図 5.2.1-3 $(k_1, k_2) = (2, 2)$ 型 : $N=26$, データ数 10000 における CNN を用いた予測精度

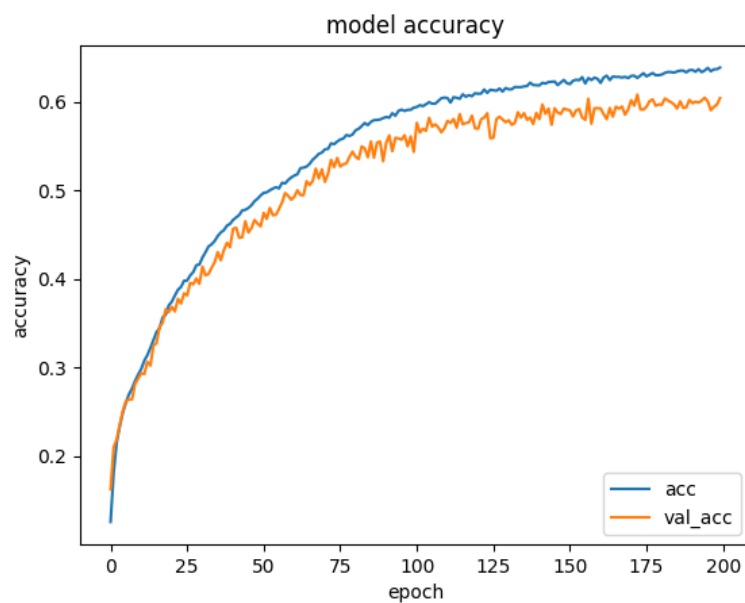


図 5.2.1-4 $(k_1, k_2) = (2, 2)$ 型 : $N=26$, データ数 50000 における CNN を用いた予測精度

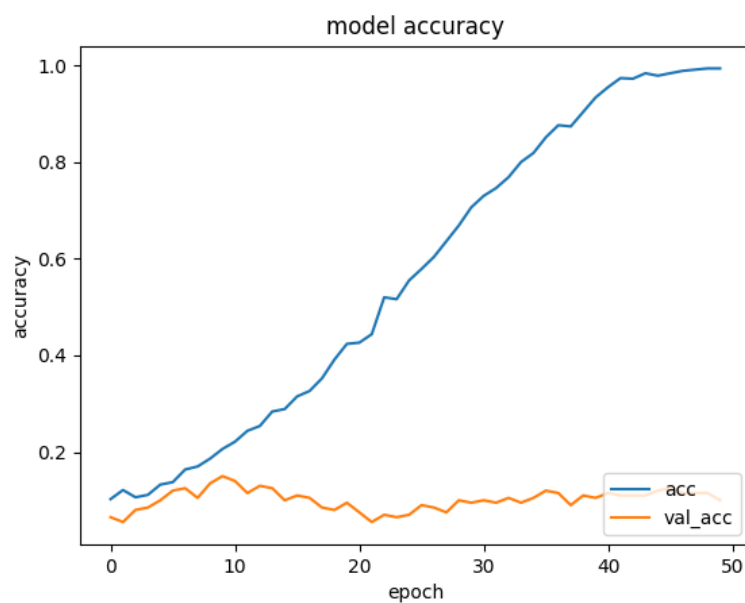


図 5.2.1-5 $(k_1, k_2) = (2, 2)$ 型 : $N=50$, データ数 1000 における CNN を用いた予測精度

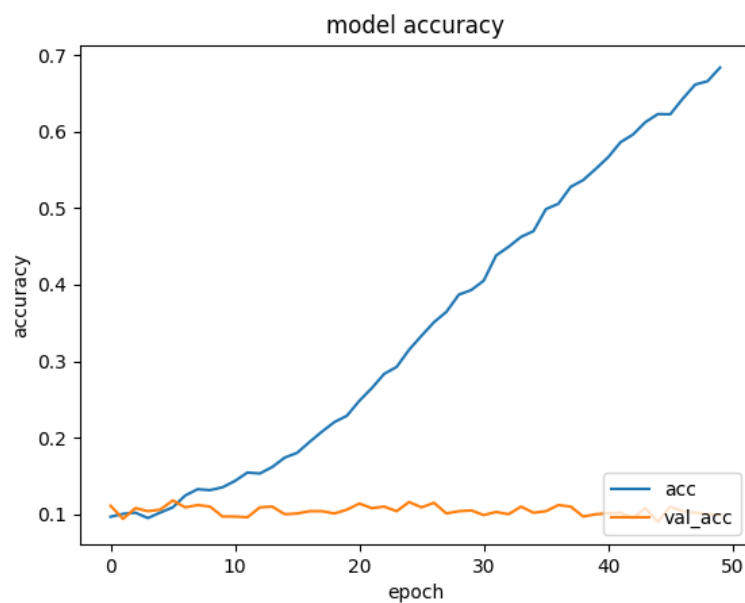


図 5.2.1-6 $(k_1, k_2) = (2, 2)$ 型 : $N=50$, データ数 5000 における CNN を用いた予測精度

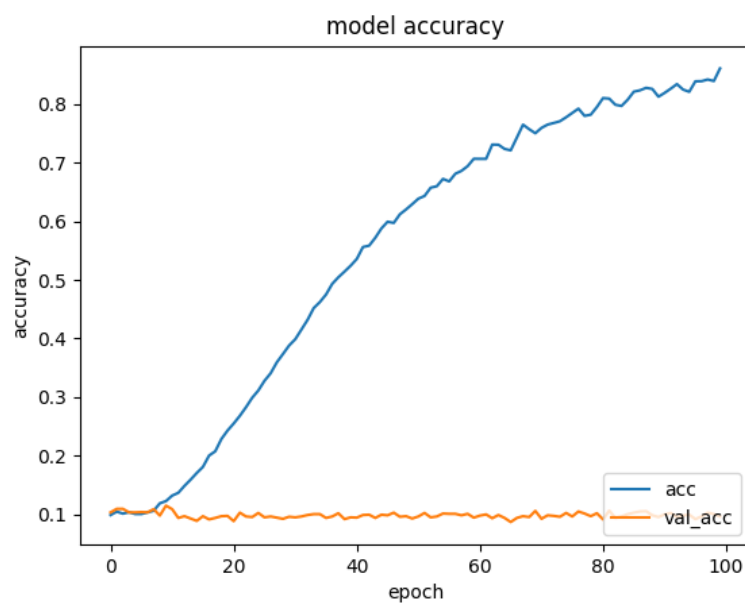


図 5.2.1-7 $(k_1, k_2) = (2, 2)$ 型 : $N=50$, データ数 10000 における CNN を用いた予測精度

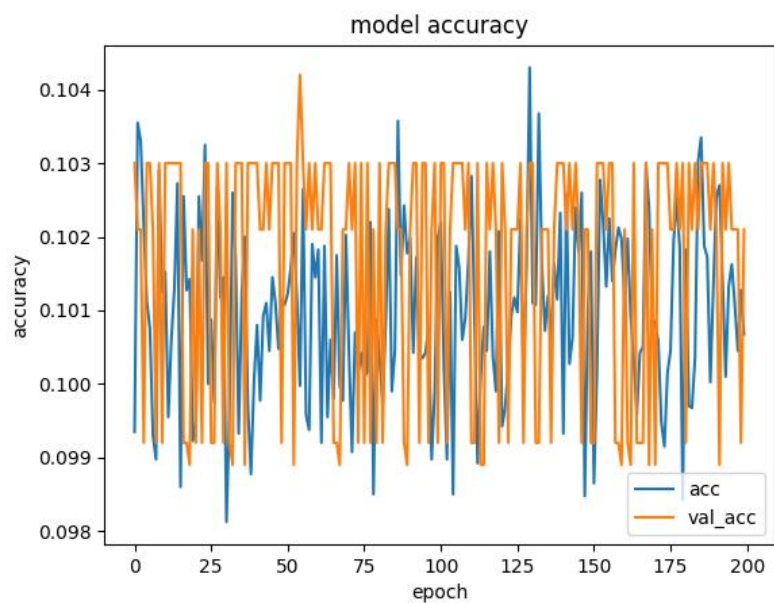


図 5.2.1-8 $(k_1, k_2) = (2, 2)$ 型 : $N=50$, データ数 50000 における CNN を用いた予測精度

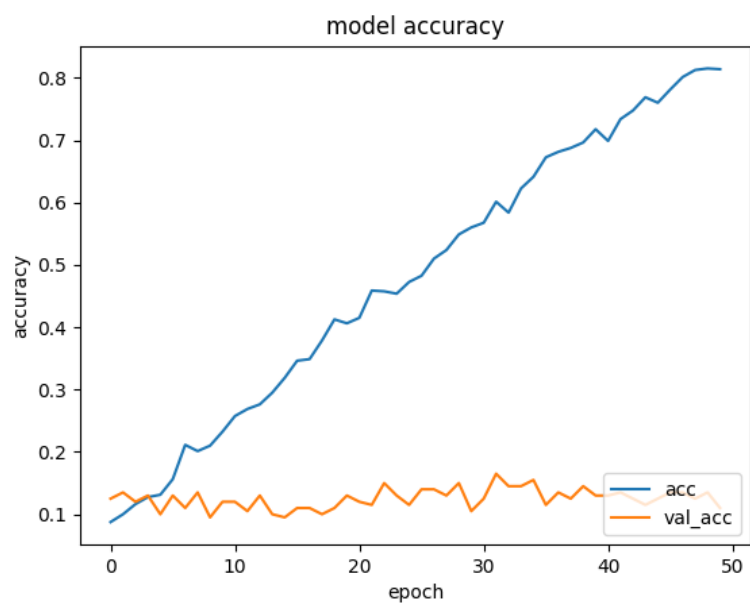


図 5.2.1-9 $(k_1, k_2) = (2, 2)$ 型 : $N=100$, データ数 1000 における CNN を用いた予測精度

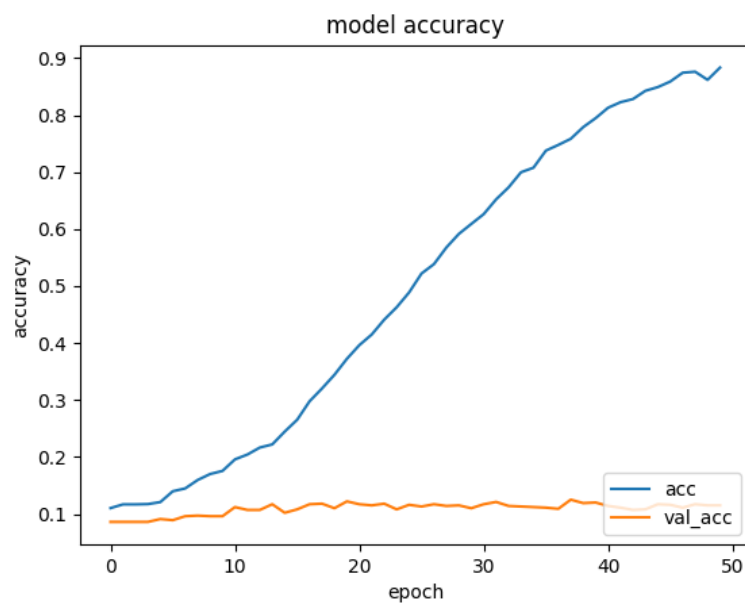


図 5.2.1-10 $(k_1, k_2) = (2, 2)$ 型 : $N=100$, データ数 5000 における CNN を用いた予測精度

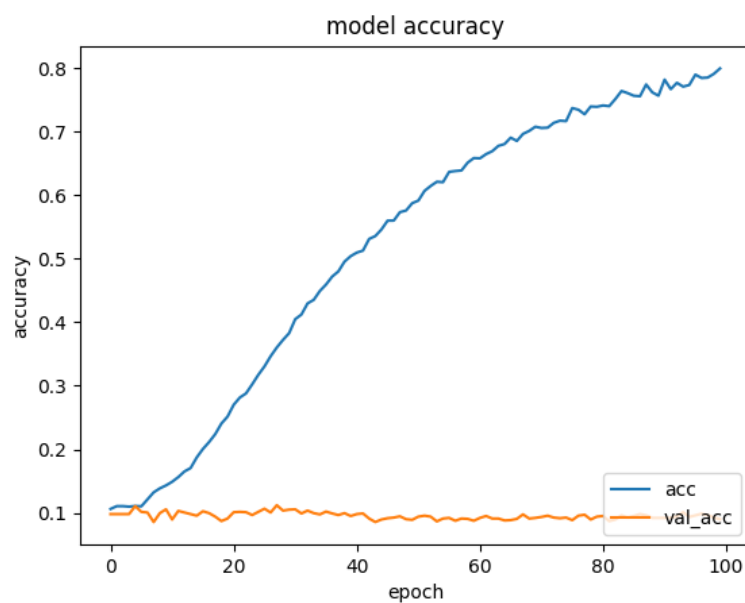


図 5.2.1-11 $(k_1, k_2) = (2, 2)$ 型 : $N=100$, データ数 10000 における CNN を用いた予測精度

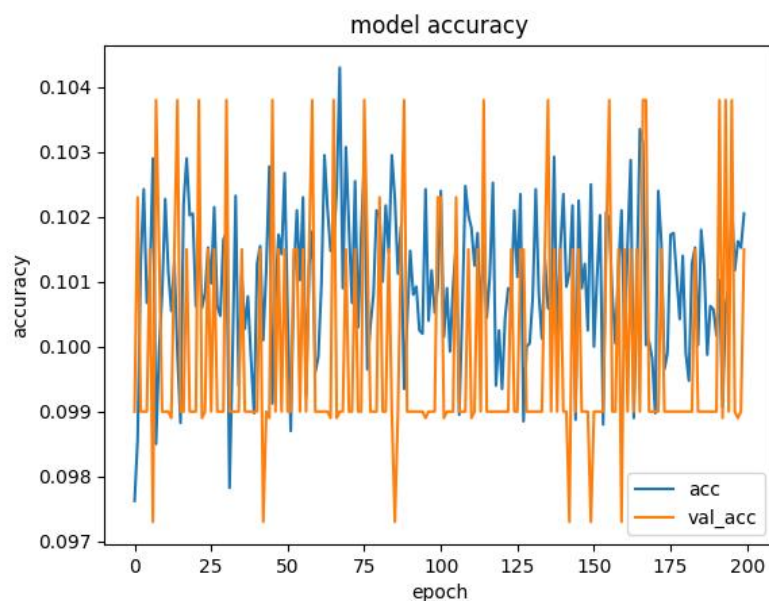


図 5.2.1-12 $(k_1, k_2) = (2, 2)$ 型 : $N=100$, データ数 50000 における CNN を用いた予測精度

5.2.2 $(k_1, k_2) = (1, 3)$ における実験結果

人間計算可能関数において、 $(k_1, k_2) = (1, 3)$ とした場合の予測精度を以下の表 5.2.2-1 および図 5.2.2-1～図 5.2.2-12 に示す。ユーザが記憶する画像枚数が 26 枚、50 枚、100 枚のいずれの場合においても、データ数の規模に関わらず、予測精度は最大でも約 15%と低い結果となった。 $(k_1, k_2) = (2, 2)$ 型のパスワードでは、ユーザが記憶する画像枚数が 26 枚、データ数 50000 の条件下で高い予測精度を示していたが、一方の $(k_1, k_2) = (1, 3)$ 型では同じ条件下でも訓練データに対する学習すら進まなかった。また、 $(k_1, k_2) = (1, 3)$ 型においても、ユーザが記憶する画像枚数が増加するにつれて、予測精度がわずかに低下する傾向が見られた。

データ数\画像枚数	$N=26$	$N=50$	$N=100$
1000	0.1521	0.1583	0.1557
5000	0.1184	0.1202	0.1309
10000	0.1160	0.1182	0.1125
50000	0.1055	0.1024	0.1085

表 5.2.2-1 $(k_1, k_2) = (1, 3)$ 型における埋め込み層および CNN を用いた予測精度

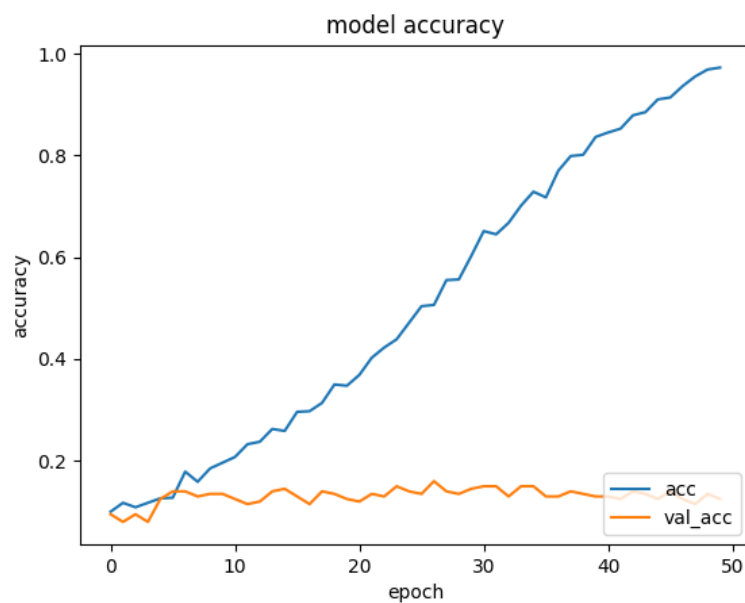


図 5.2.2-1 $(k_1, k_2) = (1, 3)$ 型 : $N=26$, データ数 1000 における CNN を用いた予測精度

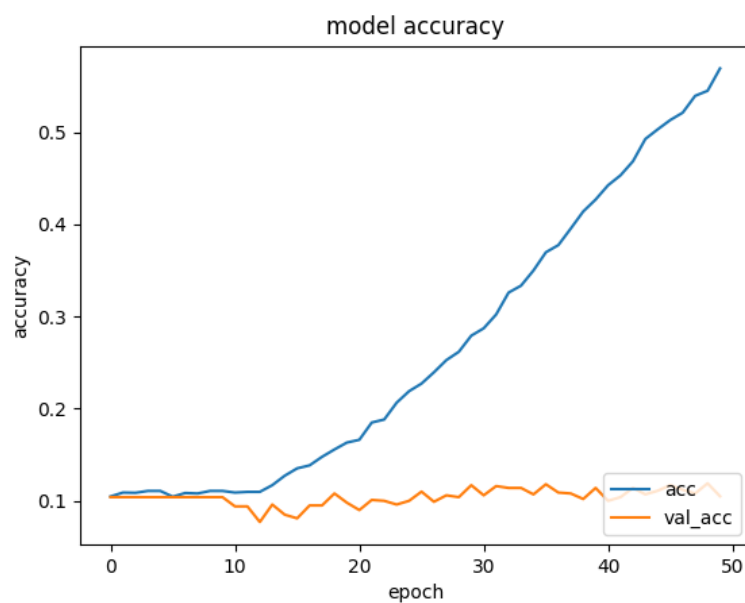


図 5.2.2-2 $(k_1, k_2) = (1, 3)$ 型 : $N=26$, データ数 5000 における CNN を用いた予測精度

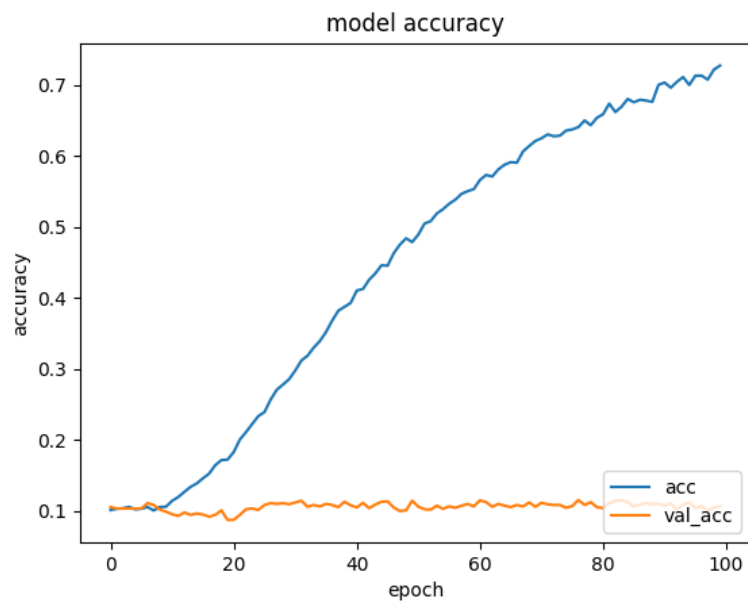


図 5.2.2-3 $(k_1, k_2) = (1, 3)$ 型 : $N=26$, データ数 10000 における CNN を用いた予測精度

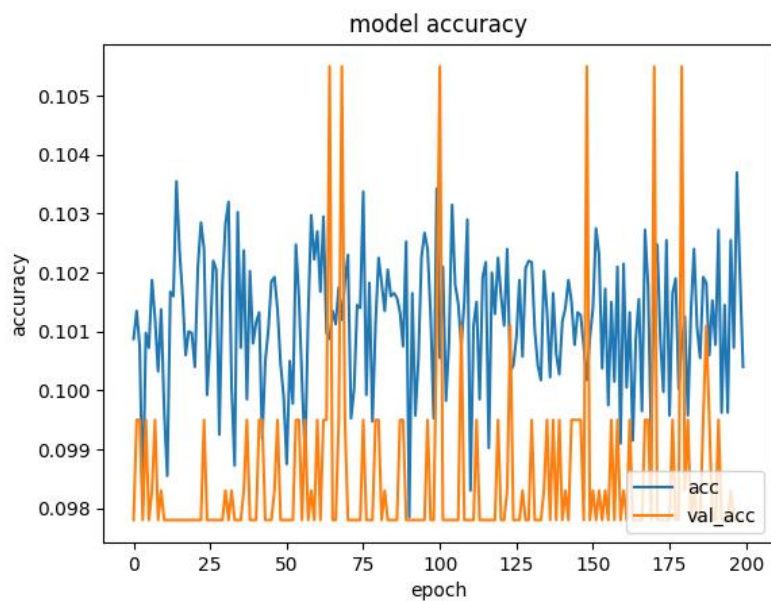


図 5.2.2-4 $(k_1, k_2) = (1, 3)$ 型 : $N=26$, データ数 50000 における CNN を用いた予測精度

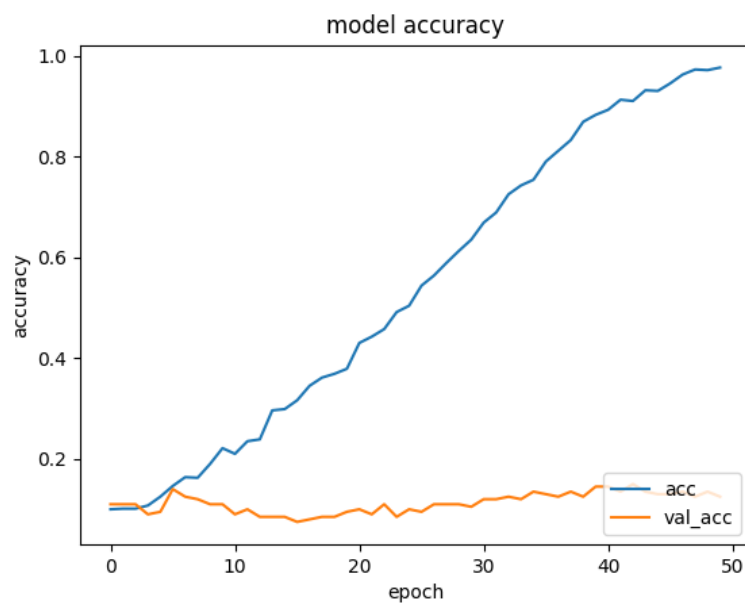


図 5.2.2-5 $(k_1, k_2) = (1, 3)$ 型 : $N=50$, データ数 1000 における CNN を用いた予測精度

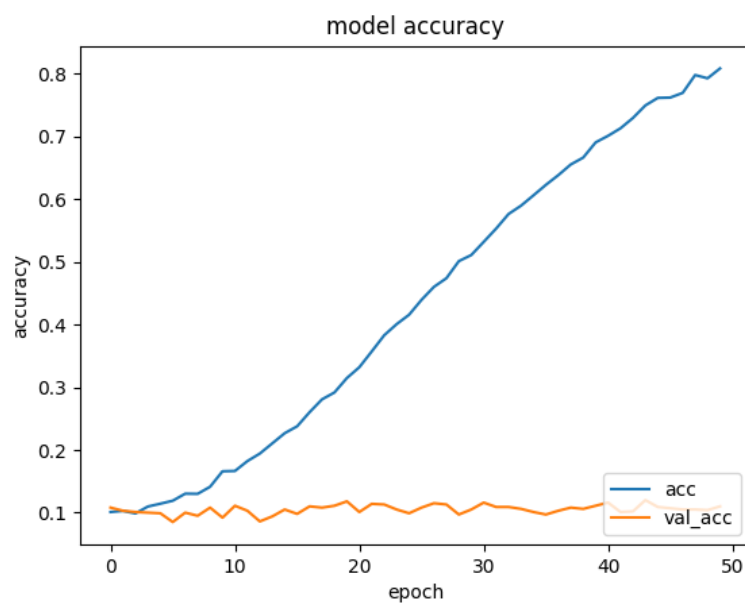


図 5.2.2-6 $(k_1, k_2) = (1, 3)$ 型 : $N=50$, データ数 5000 における CNN を用いた予測精度

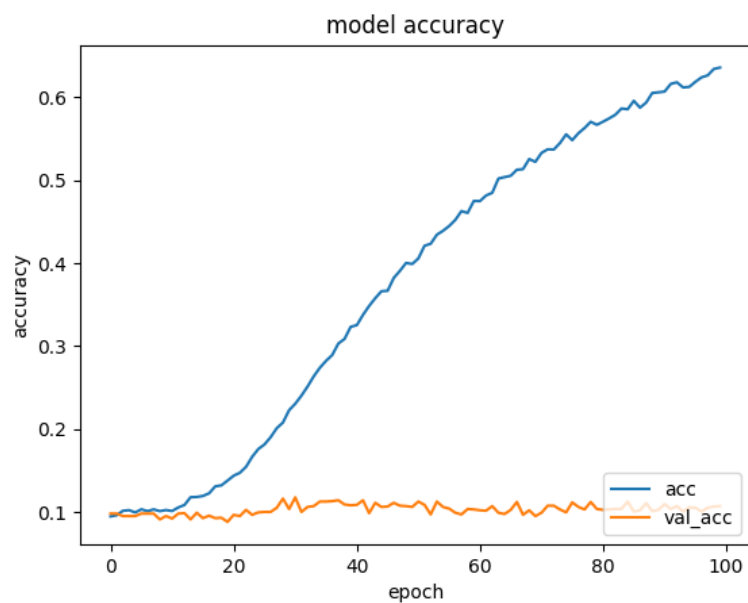


図 5.2.2-7 $(k_1, k_2) = (1, 3)$ 型 : $N=50$, データ数 10000 における CNN を用いた予測精度

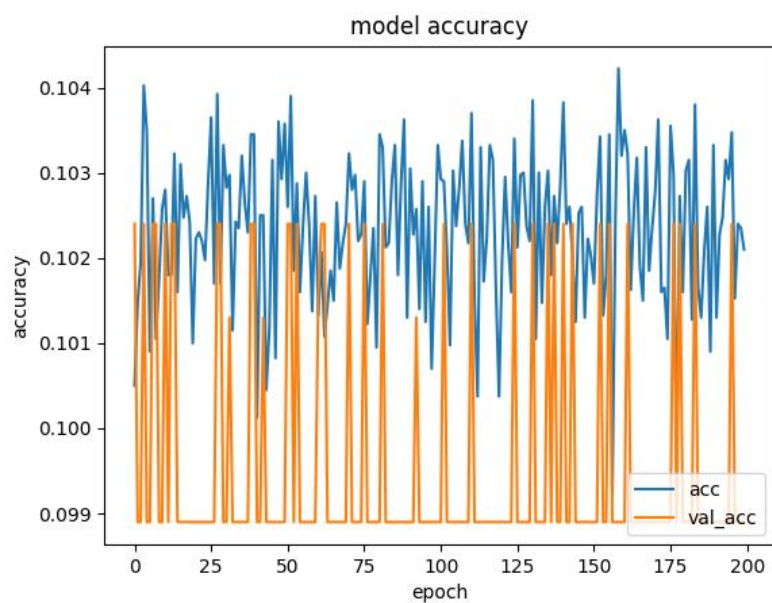


図 5.2.2-8 $(k_1, k_2) = (1, 3)$ 型 : $N=50$, データ数 50000 における CNN を用いた予測精度

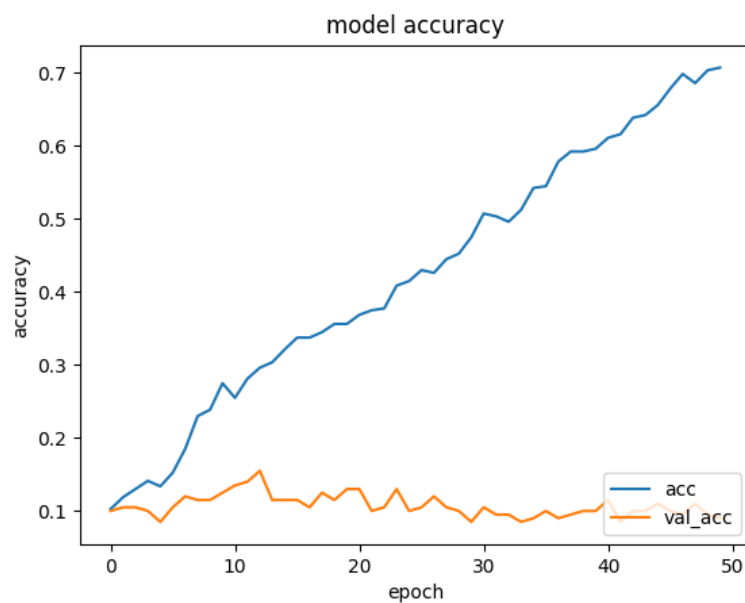


図 5.2.2-9 $(k_1, k_2) = (1, 3)$ 型 : $N=100$, データ数 1000 における CNN を用いた予測精度

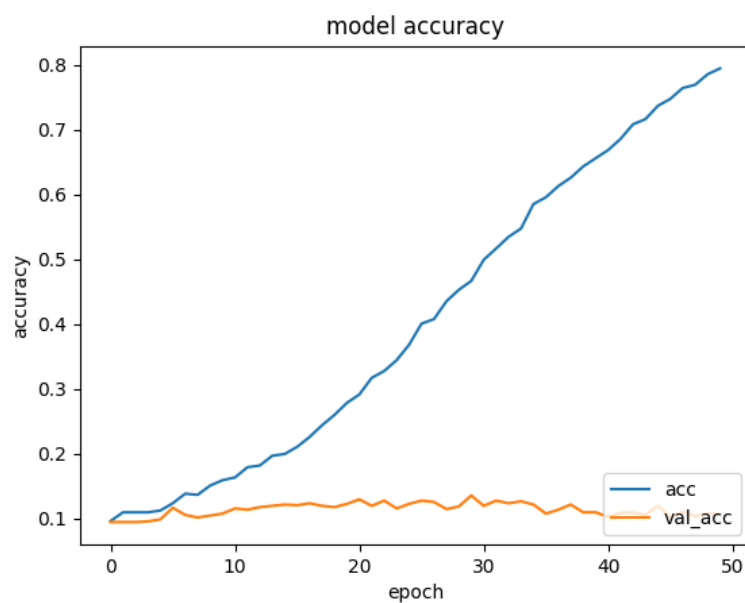


図 5.2.2-10 $(k_1, k_2) = (1, 3)$ 型 : $N=100$, データ数 5000 における CNN を用いた予測精度

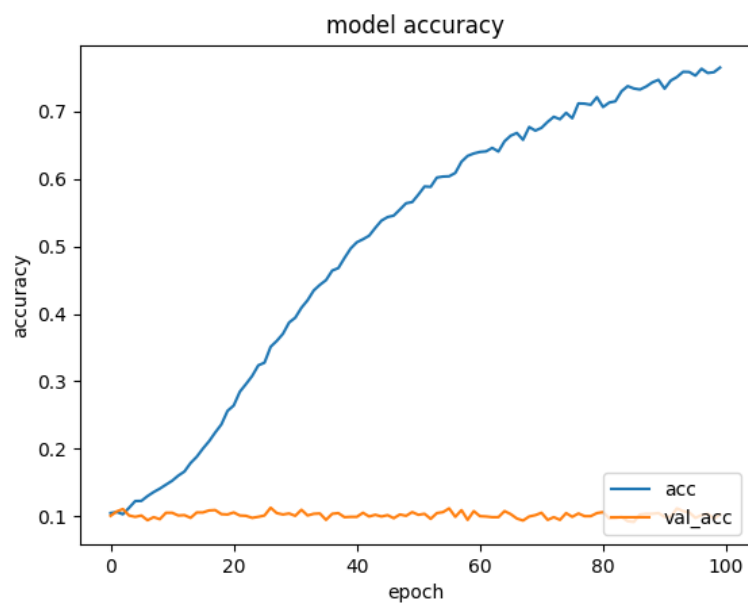


図 5.2.2-11 $(k_1, k_2) = (1, 3)$ 型 : $N=100$, データ数 10000 における CNN を用いた予測精度

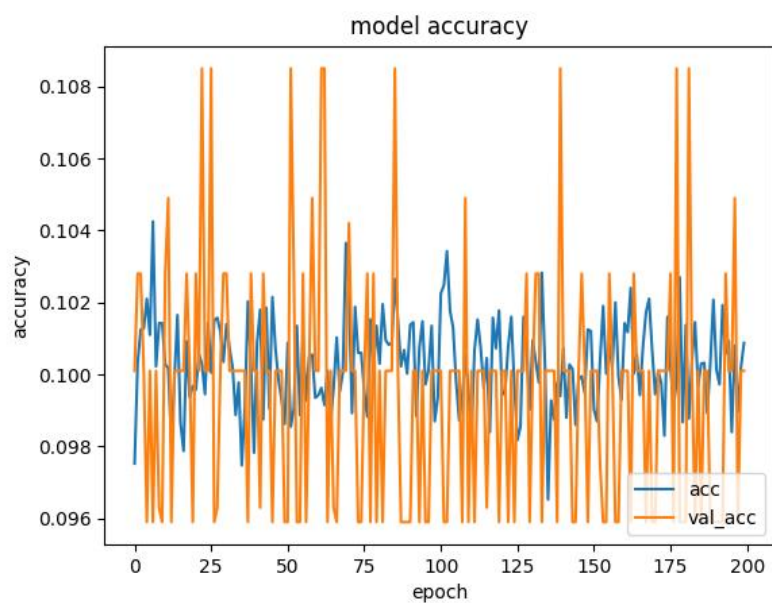


図 5.2.2-12 $(k_1, k_2) = (1, 3)$ 型 : $N=100$, データ数 50000 における CNN を用いた予測精度

第 6 章

考察

本研究では、 (k_1, k_2) の異なるパラメータ設定に基づいた人間計算可能なパスワードに対して、CNN による予測精度を検証した。その結果、 $(k_1, k_2) = (2, 2)$ 型のパスワードでは、ユーザが記憶する画像枚数が 26 枚のときに最大で 60.84% と比較的高い予測精度が達成された。一方、 $(k_1, k_2) = (1, 3)$ 型のパスワードでは、最大でも 15.83% と低い予測精度にとどまり、予測が進まない現象が観察された。特に、 $(k_1, k_2) = (1, 3)$ 型においては、 $(k_1, k_2) = (2, 2)$ 型で高い予測精度を示したときと同じ条件下でも、訓練データに対する学習すら進まず、予測精度が低迷する現象が見られた。このことから、 $(k_1, k_2) = (2, 2)$ 型よりも $(k_1, k_2) = (1, 3)$ 型の方が安全である可能性を実験的に示すことができたと言える。

また、ユーザが記憶する画像枚数が 26 枚、50 枚、100 枚の場合において、いずれの (k_1, k_2) 型でもデータ数が小さくなるにつれ過学習を起こす割合が高くなることが確認された。この過学習の原因としては、モデルの表現能力過多が考えられる。訓練データに対してニューラルネットワークの層数やパラメータ数が多く、モデルが訓練データに過剰に適合してしまうため、未知のデータに対する汎化性能が向上しないことが示唆される。

一方で、データ数が大きくなると、予測精度が安定しない傾向が見られた。これは、訓練データに対してモデルの表現能力が不足しており、訓練データのパターンを十分に学習できないこと（アンダーフィッティング）が原因と考えられる。訓練データに対する予測精度が安定しないことは、モデルがデータのパターンを捉えきれていないことを示しており、今後はモデルの表現能力をデータ数に合わせて調整する必要がある。

さらに、 $(k_1, k_2) = (2, 2)$ 型において、ユーザが記憶する画像枚数が増えると、予測精度は低下する傾向が見られた。記憶する画像枚数が増えることで、機械学習による検索空間、すなわちチャレンジとして選ばれる画像の組み合わせ数が増加し、これが予測の困難さを引き起こしていると考えられる。

総じて、人間計算可能なパスワードに対する機械学習の予測精度は、人間計算可能関数のパラメータ設定、ユーザが記憶する画像枚数、データ数、および機械学習モデルの表現能力によって大きく影響を受けることが明らかとなった。今後は、過学習やアンダーフィッティングを防ぐために、データ数やモデルの複雑さのバランスを調整することが重要であるとともに、ユーザの記憶に依存する要素を適切に設定することが求められる。

第7章

結論

本研究では、異なるパラメータ設定 (k_1, k_2) に基づく人間計算可能なパスワードの安全性を、CNN を用いた予測精度の観点から評価した。その結果、 $(k_1, k_2) = (2, 2)$ 型のパスワードでは比較的高い予測精度（最大 60.84%）を示した一方、 $(k_1, k_2) = (1, 3)$ 型のパスワードでは予測精度が低く（最大 15.83%）、学習が進まない現象が観察された。

また、データ数が少ない場合には過学習が発生しやすく、本研究で使用したモデルの表現能力は過剰であることが示唆された。一方、データ数が増加するとアンダーフィッティングにより予測精度が安定しない傾向が見られた。さらに、ユーザが記憶する画像枚数が増えるほど、機械学習による検索空間が広がることで予測精度が低下することも確認された。

以上の結果から、本研究では、人間計算可能なパスワードに対する機械学習の予測精度は、人間計算可能関数のパラメータ設定、ユーザが記憶する画像枚数、データ数、および機械学習モデルの表現能力に大きく依存することを明らかにした。

今後の課題として、過学習やアンダーフィッティングを抑制するために、データ数とモデルの複雑さの適切なバランスを取ることが重要である。また、ユーザの記憶に依存する要素の影響を考慮し、適切な設計を行うことで、より実用的で安全性の高い人間計算可能なパスワードの構築が求められる。

謝辞

本研究を進めるにあたり、ご多忙にも関わらず熱心にご指導いただきました櫻井幸一教授に心から感謝いたします。また、研究にご協力いただき、多数の助言をいただきました櫻井研究室の皆様にも心から感謝いたします。

参考文献

- [1] Manuel Blum, Santosh Vempala. The complexity of human computation via a concrete model with an application to passwords. Proceedings of the National Academy of Sciences. July 2017.
- [2] Jeremiah Blocki, Manuel Blum, Anupam Datta, Santosh Vempala. Towards Human Computable Passwords. Innovations in Theoretical Computer Science (ITCS). January 2017.
- [3] Yushuo Guan, Yuanxing Zhang, Lin Chen, Kaigui Bian. A Neural Attack Model for Cracking Passwords in Adversarial Environments. IEEE International Conference on Communications in China (ICCC). 2019. 183-188.
- [4] 村田 壱生, 人間計算可能なパスワード認証に対する深層学習による安全性評価, 九州大学理学部物理学科情報理学コース卒業論文, 2022.
- [5] 丸野 美矩人, 人間計算可能なパスワードに対する長短期記憶ニューラルネットワークによる安全性評価, 九州大学工学部電気情報工学科卒業論文, 2023.
- [6] 池田 悠登, 人間計算可能なパスワード認証に対する埋め込みを前処理とする双方向長短期記憶ニューラルネットワークによる安全性評価, 九州大学工学部電気情報工学科卒業論文, 2024.
- [7] 斎藤 康毅, ゼロから作る Deep Learning –Python で学ぶディープラーニングの理論と実装, オーム社, 2016.
- [8] 斎藤 康毅, ゼロから作る Deep Learning 2 –自然言語処理編, オーム社, 2018.
- [9] MathWorks, wordEmbeddingLayer –深層学習ニューラルネットワーク用の単語埋め込み層, <https://jp.mathworks.com/help/textanalytics/ref/nnet.cnn.layer.wordembeddinglayer.html>
- [10] MathWorks, ディープラーニング (深層学習) とは –スマートエンジニアリングシステム設計へのディープラーニング (深層学習) の適用, <https://jp.mathworks.com/discovery/deep-learning.html>
- [11] 都築 勇祐, RNN と LSTM (Long Short Term Memory) の違いと特徴, 2019, <https://www.acceluniverse.com/blog/developers/2019/07/lstm.html>
- [12] AIsmiley, 自然言語処理に使われる LSTM とは? –RNN との違いや特徴を紹介, 2024, https://aismiley.co.jp/ai_news/lstm/
- [13] MathWorks, 畳み込みニューラルネットワークとは –これだけは知っておきたい 3 つのこと, <https://jp.mathworks.com/discovery/convolutional-neural-network.html>
- [14] AIsmiley, 画像認識でよく聞く「CNN」とは? –仕組みや特徴を 1 から解説, 2024, https://aismiley.co.jp/ai_news/cnn/

付録：プログラムコード

【computable_password_generator.py】

```
import numpy as np
import pandas as pd

class ComputablePasswordGenerator:
    # 人間計算可能なパスワードの流出データを模したデータを自動生成する関数群
    # このクラスの関数を実行すると、人間計算可能なパスワードを模したデータが生成され、csv
    # ファイルとして保存される
    # 外部のプログラムから呼び出すときは、from computable_password_generator import
    # ComputablePasswordGenerator として import を行う

    # この関数群の呼び出しには int: データ数 を引数として与える
    # この関数群は pandas.DataFrame を return する
    class Utils:
        @staticmethod
        def sgm(n:int) -> np.ndarray:
            sgm = np.random.randint(0,10,n)
            return sgm

    class GeneratorWithMetadata:
        def __init__(self, generator, name :str):
            self.generator = generator
            self.name = name

        @staticmethod
        def list_generators() -> list:
            generators = []
            generators.append(ComputablePasswordGenerator.GeneratorWithMetadata(ComputablePas
            swordGenerator.s_x, "s_x"))
            generators.append( ComputablePasswordGenerator.GeneratorWithMetadata( ComputableP
            asswordGenerator.func_13, "func_13" ) )
            return generators

        @staticmethod
        def s_x(datasize :int) -> np.ndarray:
```

```

N = 26
result = []
sgm = ComputablePasswordGenerator.Utils.sgm(N)
for row in range( datasize ):
    X = np.random.randint(0,N,14)
    S_X = np.zeros(14,dtype=int)
    for k in range(14):
        S_X[k] = sgm[X[k]]
    mid = ( S_X[10] + S_X[11] ) % 10
    Z = ( S_X[12] + S_X[13] + S_X[int(mid)] ) % 10
    row = np.append(X, Z)
    result.append(row)
table_array = np.array(result)
return pd.DataFrame(table_array, columns=["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
"X8", "X9", "X10", "X11", "X12","X13", "Z"])

@staticmethod
# (k1,k2) = (1,3)
def func_13( datasize: int ) -> np.ndarray:
    N_user_memory = 26
    result = []
    sgm = ComputablePasswordGenerator.Utils.sgm( N_user_memory )
    for row in range( datasize ):
        challenge_idx = np.random.randint( 0, N_user_memory, 14 )
        X = np.zeros( 14, dtype = int )
        for k in range( 14 ):
            X[k] = sgm[challenge_idx[k]]
        j = X[10] % 10
        Z = ( X[int(j)] + X[11] + X[12] + X[13] ) % 10
        row = np.append( challenge_idx, Z )
        result.append( row )
    table_array = np.array( result )
    return pd.DataFrame( table_array, columns = ["X0", "X1", "X2", "X3", "X4", "X5", "X6", "X7",
"X8", "X9", "X10", "X11", "X12", "X13", "Z"] )

```

【utils.py】

```
from matplotlib import pyplot
from tensorflow import keras
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

class Utils:
    # グラフのプロットを行う関数
    # keras.model.fit.history および画像の保存名を引数として取る
    @staticmethod
    def plot_history(history :str, name :str) -> None:
        # 学習時の訓練データおよび検証データに対する正解率を表示する
        pyplot.clf()
        pyplot.plot(history['accuracy'])
        pyplot.plot(history['val_accuracy'])
        pyplot.title('model accuracy')
        pyplot.xlabel('epoch')
        pyplot.ylabel('accuracy')
        pyplot.legend(['acc', 'val_acc'], loc='lower right')
        pyplot.savefig("outputs/" + name + '_accuracy.png')

        # 学習時の訓練データおよび検証データに対する損失関数の値を表示する
        pyplot.clf()
        pyplot.plot(history['loss'])
        pyplot.plot(history['val_loss'])
        pyplot.title('model loss')
        pyplot.xlabel('epoch')
        pyplot.ylabel('loss')
        pyplot.legend(['loss', 'val_loss'], loc='lower right')
        pyplot.savefig("outputs/" + name + '_loss.png')
        return None

    @staticmethod
    def split_to_train_and_valid(generated_passwords :pd.DataFrame) -> (np.ndarray, np.ndarray,
np.ndarray, np.ndarray):
        generated_passwords = generated_passwords.sample(frac=1).reset_index(drop=True)
```

```
x = generated_passwords.drop(labels = ["Z"],axis = 1)
y = generated_passwords["Z"]

# 説明変数・目的変数をそれぞれ訓練データ・テストデータに分割
x_train,x_test,y_train,y_test = train_test_split(x, y, test_size=0.2)

y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)

return x_train, x_test, y_train, y_test

class LossHistory(keras.callbacks.Callback):
    def on_train_begin(self, logs={}):
        self.losses = []

    def on_batch_end(self, batch, logs={}):
        self.losses.append(logs.get('loss'))
```

【models.py】

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Input, Embedding, Conv1D
import pandas as pd
import numpy as np
import tensorflow as tf

class Models:
    class ModelWithMetadata:
        def __init__(self, model, name :str, batch_size :int, epochs :int, required_data_size :int):
            self.model = model
            self.name = name
            self.batch_size = batch_size
            self.epochs = epochs
            self.required_data_size = required_data_size

        def resharper(self, df :pd.DataFrame) -> (np.ndarray):
```

```
if self.name.find("lstm") != -1:
    return df.to_numpy().reshape(df.shape[0], df.shape[1], 1)
return df

@staticmethod
def list_models() -> list:
    models = []
    models.append(Models.ModelWithMetadata(Models.embed_cnn(), "cnn_with_embedding",
    25, 200, 50000))
    return models

@staticmethod
# 埋め込み層 & 畳み込みニューラルネットワーク
def embed_cnn() -> Model:
    N = 26
    # 入力層
    inputs = Input(shape=(14, ))
    # 埋め込み層
    embedding = Embedding(input_dim=N, output_dim=3)(inputs)
    # 畳み込み層
    conv = Conv1D(filters=32, kernel_size=3, activation="relu")(embedding)
    conv = Conv1D(filters=64, kernel_size=3, activation="relu")(conv)
    conv = Flatten()(conv)
    # 全結合層
    dense = Dense(30, activation="relu")(conv)
    dense = Dense(30, activation="relu")(dense)
    dense = Dense(30, activation="relu")(dense)
    # 出力層
    outputs = Dense(10, activation="softmax")(dense)
    # モデル定義
    model = Model(inputs=inputs, outputs=outputs)
    model.compile(loss="categorical_crossentropy", optimizer="Adam", metrics=["accuracy"])
    model.summary()
    return model
```

【batch_run.py】

```
from computable_password_generator import ComputablePasswordGenerator
from utils import Utils
from models import Models
import os
import traceback

iter = 0
try:
    os.mkdir("outputs/{}".format(iter))
except:
    pass
for model in Models.list_models():
    print("Runnning model: {}".format(model.name))
    for generator in ComputablePasswordGenerator.list_generators():
        try:
            print("Testing: generator: {}, model: {}".format(generator.name, model.name))
            print("Figure name: {}".format(generator.name + "_" + model.name))
            generated_passwords = generator.generator(model.required_data_size)
            x_train, x_test, y_train, y_test = Utils.split_to_train_and_valid(generated_passwords)
            x_train = model.resharper(x_train)
            x_test = model.resharper(x_test)
            model.model.fit(x_train, y_train, batch_size=model.batch_size, epochs=model.epochs,
                           verbose=1, validation_data=(x_test, y_test), callbacks=[tensorboard])
            history = model.model.history.history
            Utils.plot_history(history, "{}"/".format(iter) + generator.name + "_" + model.name)
        except Exception as e:
            print("Error: generator: {}, model: {}".format(generator.name, model.name))
            print(e)
            traceback.print_exc()
            continue
```